Avaliação de LLMs na redocumentação de regras de negócio em programas COBOL

Matheus Martins da Cruz



Universidade Federal de Uberlândia
Faculdade de Computação
Bacharelado em Ciência da Computação

Uberlândia - MG
2026

Matheus Martins da Cruz

Avaliação de LLMs na redocumentação de regras de negócio em programas COBOL

Trabalho de Conclusão de Curso apresentado à Faculdade de Computação da Universidade Federal de Uberlândia, Minas Gerais, como requisito exigido parcial à obtenção do Título de Bacharel em Ciência da Computação

Área de concentração: Ciência da Computação

Orientador: Prof. Marcelo de Almeida Maia

Uberlândia - MG
2026

Resumo

Sistemas legados desenvolvidos em *Common Business-Oriented Language* (*COBOL*) permanecem essenciais para as operações de organizações dos setores financeiro e governamental. Contudo, a manutenção e a modernização dessas arquiteturas são dificultadas pela escassez de documentação atualizada. Este trabalho investiga a eficácia do uso de *Large Language Models* (*LLMs*) na geração de documentação de regras de negócio a partir de código *COBOL*, com o intuito de subsidiar processos de manutenção e modernização com maior confiabilidade. Para isso, propõe-se uma metodologia multicritério baseada em regras de negócio atômicas. Essa abordagem avalia a documentação gerada por correspondência semântica com um gabarito (*ground truth*), em vez de comparação textual literal, para evitar a penalização de reformulações válidas. O método será aplicado a quatro programas da *CICS Banking Sample Application* (*CBSA*), sistema bancário de referência da *IBM*, com um gabarito de 37 regras de negócio. Os modelos *Claude Sonnet* 5.5 e *GPT-6* Sol serão comparados com três estratégias de *prompt* (*zero-shot*, *few-shot* e *chain-of-thought*), e cada item das respostas será classificado como fora do escopo, alucinação, reformulação ou enriquecimento. Espera-se, com a execução deste método, mensurar a precisão, a revocação (*recall*), o *F1 score* e a taxa de alucinação da documentação gerada, e comparar os resultados obtidos aos valores reportados na literatura correlata. Essa estrutura visa contribuir para uma avaliação mais criteriosa e replicável do uso de *LLMs* na modernização de sistemas legados.

Palavras-chave: Sistemas legados, *COBOL*, Modelos de linguagem de grande escala, Redocumentação, Regras de Negócio.

Abstract

Legacy systems developed in Common Business-Oriented Language (COBOL) remain essential to the operations of organizations in the financial and government sectors. However, their maintenance and modernization are hindered by the scarcity of up-to-date documentation. This work investigates the effectiveness of Large Language Models (LLMs) in generating business rule documentation from COBOL code, to support maintenance and modernization processes with greater reliability. To this end, a multi-criteria methodology based on atomic business rules is proposed. The approach evaluates the generated documentation through semantic correspondence with a ground truth, rather than literal textual comparison, to avoid unfair penalization of valid rephrasing. The method will be applied to four programs of the CICS Banking Sample Application (CBSA), an IBM reference banking system, with a ground truth of 37 business rules. The Claude Sonnet 5.5 and GPT-6 Sol models will be compared using three prompting strategies (zero-shot, few-shot, and chain-of-thought), and each item of the responses will be classified as out of scope, hallucination, rephrasing, or enrichment. By executing this method, it is expected to measure the precision, recall, F1 score, and hallucination rate of the generated documentation, and to compare the results with values reported in the related literature. This structure aims to contribute to a more rigorous and replicable evaluation of the use of LLMs in legacy system modernization.

Keywords: Legacy systems, COBOL, Large language models, Redocumentation, Business Rule.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Em conformidade com o Código de Conduta para Autores em Publicações (SOCIEDADE BRASILEIRA DE COMPUTAÇÃO, 2024), declara-se o uso de ferramentas de inteligência artificial generativa na preparação deste trabalho. Foram empregadas a ferramenta Claude (Anthropic), com o modelo Claude Opus 5.5, e a ferramenta Gemini Notebook (Google), antigo NotebookLM, com o modelo Gemini 3.1 Pro.

O Claude foi empregado como apoio em: (i) etapas da pesquisa, como a redação preliminar das regras do gabarito, a especificação dos *prompts*, as regras de contagem e os *scripts* de preparação dos dados; (ii) assistência à redação de trechos desta monografia, a partir de conteúdo, dados e decisões definidos pelo autor; (iii) conferência das referências bibliográficas; e (iv) geração do código das figuras. O Gemini Notebook foi empregado em: (i) redação do texto final dos três *prompts*, a partir da especificação elaborada pelo autor; e (ii) revisão linguística e estilística do texto.

A concepção do estudo, o desenho experimental, a validação de cada regra do gabarito no código-fonte, a execução das conversas com os modelos avaliados, a classificação das respostas e as conclusões são de responsabilidade do autor. O uso dessas ferramentas na pesquisa e o seu efeito sobre a validade dos resultados são discutidos na Seção 4.8. Nenhum conteúdo gerado por inteligência artificial foi usado como fonte bibliográfica, e o autor revisou integralmente o texto e assume total responsabilidade pelo seu conteúdo.

Lista de ilustrações

Figura 1 – Formato fixo de referência de um programa *COBOL* 16
Figura 2 – Avaliação de sistemas legados 18
Figura 3 – Visão geral do procedimento de avaliação 27
Figura 4 – Exemplo de par do gabarito e da regra atômica correspondente 31

Lista de tabelas

Tabela 1 – Comparativo dos trabalhos relacionados 25
Tabela 2 – Programas selecionados para a avaliação 29
Tabela 3 – Extração e classificação dos pares do gabarito 32
Tabela 4 – Linhas removidas do código enviado aos modelos 34

Lista de siglas

ABEND Término anormal da tarefa - *Abnormal End*

ACL Association for Computational Linguistics

ACM Association for Computing Machinery

ALC Código em Linguagem Assembly - *Assembly Language Code*

ANSI American National Standards Institute

API Interface de Programação de Aplicações - *Application Programming Interface*

ASCII Código Padrão Americano para o Intercâmbio de Informação - *American Standard Code for Information Interchange*

ASE Automated Software Engineering

ASEW Automated Software Engineering Workshops

BERT Bidirectional Encoder Representations from Transformers

CASE Engenharia de Software Auxiliada por Computador - *Computer-Aided Software Engineering*

CBSA CICS Banking Sample Application

CICS Customer Information Control System

COBOL Common Business-Oriented Language

EASE International Conference on Evaluation and Assessment in Software Engineering

GPT Generative Pre-trained Transformer

IA Inteligência Artificial

IBM International Business Machines

ICSE International Conference on Software Engineering

IEEE Institute of Electrical and Electronics Engineers

LLaMA Large Language Model Meta AI

LLM Modelo de Linguagem de Grande Escala - *Large Language Model*

LLM4Code International Workshop on Large Language Models for Code

MCQA Resposta a Perguntas de Múltipla Escolha - *Multiple-Choice Question Answering*

MUMPS Massachusetts General Hospital Utility Multi-Programming System

PL/I Programming Language One

RAG Geração Aumentada por Recuperação - *Retrieval-Augmented Generation*

SHA Algoritmo de Hash Seguro - *Secure Hash Algorithm*

SQL Linguagem de Consulta Estruturada - *Structured Query Language*

Sumário

1 INTRODUÇÃO 11
1.1 Motivação 11
1.2 Problema 12
1.3 Hipótese 12
1.4 Objetivos 13
1.5 Contribuições 13
1.6 Organização da Monografia 14

2 FUNDAMENTAÇÃO TEÓRICA 15
2.1 Sistemas Legados e *COBOL* 15
2.1.1 Definições fundamentais 15
2.1.2 Breve histórico 16
2.1.3 Avanços recentes da área 17
2.1.4 Exemplos relevantes 18
2.2 *Large Language Models (LLMs)* 19
2.2.1 Definições fundamentais 19
2.2.2 Breve histórico 20
2.2.3 Avanços recentes da área 20
2.2.4 Exemplos relevantes 21

3 TRABALHOS RELACIONADOS 22
3.1 Extração de regras de negócio em *COBOL* 22
3.2 Documentação e compreensão de código legado 23
3.3 Comparação entre os trabalhos 24

4 MÉTODO 26
4.1 Visão geral 26
4.2 Base de dados 28

4.3 Construção do gabarito 30
4.4 Estratégias de *prompt* 32
4.5 Preparação da entrada e execução 33
4.6 Critérios de validação e correção 35
4.7 Passos do desenvolvimento 37
4.8 Uso de ferramentas de Inteligência Artificial 38

5 CONCLUSÃO 40

REFERÊNCIAS 41

APÊNDICES 45

APÊNDICE A – TEXTO DOS *PROMPTS* 46
A.1 Base comum aos três *prompts* 46
A.2 Fechamento comum aos três *prompts* 47
A.3 Bloco do *prompt* B (*few-shot*) 48
A.4 Bloco do *prompt* C (*chain-of-thought*) 51

Capítulo 1

Introdução

Sistemas legados caracterizam-se como *softwares* fundamentais para a continuidade dos negócios de organizações dos setores financeiro e governamental (KHEMKA; MAJUMDAR, 2025). Frequentemente, essas arquiteturas estão hospedadas em *mainframes* e são programadas em *Common Business-Oriented Language (COBOL)*. Este trabalho delimita-se ao estudo da geração automatizada de documentação de regras de negócio embutidas nesses sistemas por meio de *Large Language Models (LLMs)*, investigando em que medida tais modelos conseguem interpretar corretamente lógicas de negócio codificadas há décadas, sem acesso à documentação original. A relevância do tema decorre tanto do impacto econômico dos sistemas legados, que sustentam aproximadamente 70% das operações financeiras globais (BANDARUPALLI, 2025), quanto da crescente adoção de IA generativa como estratégia de apoio à modernização de software na indústria (IBM, 2025).

1.1 Motivação

As limitações da época em que os sistemas legados foram implementados geram expressivas dificuldades de manutenção e modernização. Essa defasagem impõe alto custo e risco na transição para novas tecnologias. Nesse cenário, a utilização de *LLMs* apresenta-se como uma ferramenta de apoio eficiente. Tais modelos auxiliam na compreensão das regras de negócio, no mapeamento do fluxo de dados e na redocumentação dos códigos. Essa automação acelera um processo que, se realizado de maneira manual, seria excessivamente lento e suscetível a falhas (RAJBHOJ et al., 2025). Paralelamente, o tema ganha destaque no meio acadêmico ao impulsionar o uso de Inteligência Artificial (IA) e *Machine Learning* na automação da Engenharia de *Software*. O relatório anual da IBM (2025) reforça esse movimento. O documento destaca investimentos em IA aplicada a cargas de trabalho de missão crítica e a capacidade de executar agentes inteligentes diretamente em plataformas *IBM Z*.

Estudos recentes começam a validar essa direção, ainda que de forma incompleta. A

geração automatizada de documentação por *LLMs* mostra-se mais segura do que a tradução direta do código-fonte. A criação de comentários explicativos em nível de linha mitiga o risco de modificações inadvertidas no sistema. No entanto, o processo ainda apresenta limitações e alucinações ao lidar com linguagens de alta complexidade estrutural, como *Assembly* (DIGGS et al., 2025). No contexto específico de linguagens de negócio, Rajbhoj et al. (2025) aplicaram engenharia de *prompt* em arquitetura *Maker-Checker* para extrair a lógica de módulos *COBOL* industriais. Os autores alcançaram 91% de precisão média. Contudo, o estudo evidenciou dificuldades persistentes com a sintaxe antiga da linguagem, sobretudo com variáveis dinâmicas e comandos de desvio de fluxo, como a instrução *GO TO*. Esses resultados, detalhados no Capítulo 3, mostram que ainda não há consenso sobre a confiabilidade da documentação gerada por *LLMs* para sistemas legados em produção. Além disso, não há uma definição clara sobre como avaliar essa confiabilidade sem penalizar reformulações válidas do conteúdo original.

1.2 Problema

O problema central desta pesquisa consiste em avaliar, de forma confiável e sem penalizar reformulações válidas, a qualidade e a fidelidade da documentação de regras de negócio gerada por *LLMs* a partir de código *COBOL* legado. Métricas tradicionais de comparação textual tendem a penalizar respostas que expressam a mesma regra de negócio com palavras diferentes, o que subestima a sua utilidade prática (ROY; FAKHOURY; ARNAOUDOVA, 2021).

Ao mesmo tempo, essas métricas podem não capturar adequadamente casos de alucinação, em que o conteúdo gerado diverge do comportamento real do código. Essa limitação metodológica justifica a proposição de uma abordagem de avaliação centrada em regras de negócio atômicas, verificadas diretamente no código-fonte, capaz de mensurar com maior precisão a efetividade dos *LLMs* nesse cenário de modernização.

1.3 Hipótese

A hipótese deste estudo é que modelos de linguagem de grande escala, especificamente o *Claude Sonnet* 5.5 e o *GPT-6* Sol, são capazes de gerar documentação de regras de negócio a partir de código *COBOL* legado. A expectativa é que o processo alcance uma precisão comparável à reportada por Rajbhoj et al. (2025), de 91%, e uma concordância próxima à obtida por Kalohia (2026), de 93%. Apesar disso, o estudo parte da premissa de que os modelos mantêm uma taxa relevante de alucinações ao interpretar lógicas de negócio não lineares, a exemplo de estruturas condicionais aninhadas e desvios de fluxo. Para investigar o tema, o trabalho busca responder às seguintes perguntas de pesquisa:

- ❏ Quais são a precisão, a revocação e o F1 da documentação gerada pelos modelos avaliados, em comparação a um gabarito construído com base em código real?
- ❏ Existe diferença relevante de desempenho entre os modelos *Claude Sonnet* 5.5 e *GPT-6* Sol na tarefa proposta?
- ❏ A estratégia de *prompt* (*zero-shot*, *few-shot* ou *chain-of-thought*) altera a precisão, a revocação e a taxa de alucinação?

Para verificar a hipótese, o experimento descrito no Capítulo 4 utiliza quatro programas da *CBSA* e um gabarito de 37 regras de negócio atômicas, conferidas no código-fonte. Cada programa é submetido aos dois modelos com as três estratégias de *prompt*, em três repetições, o que totaliza 72 conversas. Cada item das respostas é classificado como fora do escopo, alucinação, reformulação ou enriquecimento, e essas categorias permitem calcular a precisão, a revocação, o *F1 score* e a taxa de alucinação. As respostas às perguntas de pesquisa serão apresentadas e discutidas nos capítulos de Resultados e Discussão, a serem elaborados na próxima etapa da pesquisa.

1.4 Objetivos

Objetivo geral

O objetivo geral deste trabalho é investigar a eficácia e a precisão do uso de *LLMs* na geração de documentação de regras de negócio a partir de código *COBOL* de sistemas legados em produção. Com isso, busca-se subsidiar processos de manutenção e de portabilidade, para uma modernização com integridade e confiabilidade.

Objetivos específicos

- ❏ Identificar, na literatura científica e na indústria, as principais abordagens baseadas em *LLMs* aplicadas à modernização e à extração de lógicas de negócio em códigos antigos.
- ❏ Propor um procedimento de avaliação da documentação de regras de negócio gerada por *LLMs* a partir de código *COBOL*, baseado em regras atômicas e em um gabarito construído com o código real.
- ❏ Avaliar experimentalmente a qualidade da documentação gerada, mensurando a precisão, a revocação e a incidência de alucinações dos modelos utilizados durante a interpretação das lógicas de negócio.

1.5 Contribuições

Este trabalho pretende contribuir com:

- ❏ Um procedimento de geração de documentação de regras de negócio a partir de código *COBOL* legado, composto por *scripts* de preparação da entrada, publicados em repositório aberto, e por um protocolo de execução nos modelos *Claude Sonnet* 5.5 e *GPT-6* Sol;
- ❏ Um gabarito (*ground truth*) estruturado, construído a partir de código real do sistema *CICS Banking Sample Application (CBSA)* (IBM, 2023), responsável por conectar as regras de negócio diretamente aos trechos de código-fonte que as implementam;
- ❏ Uma metodologia de avaliação multicritério baseada em regras de negócio atômicas, capaz de diferenciar alucinação, enriquecimento e reformulação e de separar os itens que não são regras de negócio, de modo a mitigar uma limitação conhecida das métricas tradicionais de comparação textual.

1.6 Organização da Monografia

Esta monografia está organizada em quatro capítulos, além desta introdução. O Capítulo 2 apresenta a fundamentação teórica necessária à compreensão do trabalho, com os conceitos de sistemas legados e da linguagem *COBOL* e os fundamentos e a evolução dos *LLMs*. O Capítulo 3 discute os trabalhos relacionados, por meio da comparação de abordagens da literatura que aplicam *LLMs* à modernização e à extração de regras de negócio em sistemas legados.

O Capítulo 4 descreve o método proposto e detalha a base de dados utilizada, a construção do gabarito, os critérios de validação e classificação dos resultados, além das etapas planejadas para o desenvolvimento da pesquisa. O Capítulo 5 apresenta as conclusões desta etapa e os próximos passos da pesquisa. Os capítulos de resultados e discussão serão desenvolvidos na próxima etapa, a partir da execução do método aqui apresentado.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta os conceitos necessários à compreensão do trabalho, organizados em dois temas. A Seção 2.1 trata dos sistemas legados e da linguagem *COBOL*, incluindo a estrutura dos programas e o ambiente em que são executados. A Seção 2.2 trata dos *LLMs* e das estratégias de *prompt* comparadas neste trabalho. Cada tema é apresentado por suas definições fundamentais, seu histórico, seus avanços recentes e exemplos relevantes.

2.1 Sistemas Legados e *COBOL*

2.1.1 Definições fundamentais

A literatura de Engenharia de *Software* define sistemas legados como aplicações de base tecnológica defasada que, por desempenharem funções operacionais críticas, permanecem em uso contínuo. Sujeitos a extensos períodos de manutenção, estes sistemas frequentemente apresentam uma estrutura interna degradada. Esta característica torna os processos de evolução altamente complexos e custosos (SOMMERVILLE, 2011).

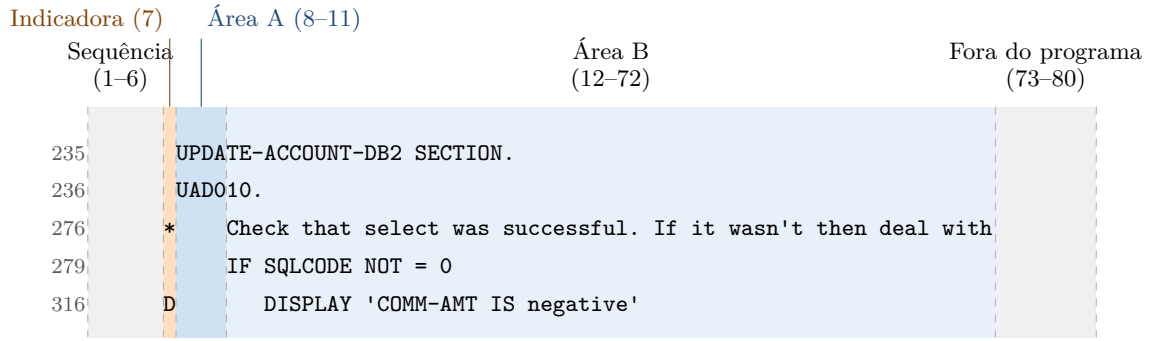
Como pilar estrutural de grande parte desses ambientes encontra-se a *Common Business-Oriented Language (COBOL)*. Desenvolvida estritamente para o meio corporativo, a linguagem inovou ao organizar dados em registros e adotar uma sintaxe semelhante à língua inglesa. Esta abordagem otimizou o processamento de transações comerciais em larga escala (SAMMET, 1978).

Um programa *COBOL* organiza-se em quatro divisões, nesta ordem: *IDENTIFICATION DIVISION*, que identifica o programa; *ENVIRONMENT DIVISION*, que descreve o ambiente computacional e os arquivos utilizados; *DATA DIVISION*, que declara os dados; e *PROCEDURE DIVISION*, que contém as instruções executáveis (IBM, 2024). A *PROCEDURE DIVISION* pode ser organizada em seções (*SECTION*) e parágrafos, blocos nomeados que servem de destino para comandos como *PERFORM* e *GO TO*. O comando *COPY* insere no programa, durante a compilação, textos mantidos em arqui-

vos separados, chamados *copybooks*, como as definições de dados compartilhadas entre programas.

No formato fixo de referência, cada coluna da linha tem uma função, como mostra a Figura 1. As colunas 1 a 6 formam a área de número de sequência, e a coluna 7 é a área indicadora: um asterisco ou uma barra nessa coluna marcam uma linha de comentário, a letra D (ou d) marca uma linha de depuração e um hífen indica a continuação da linha anterior (IBM, 2024). Os nomes de divisões, seções e parágrafos começam na área A, nas colunas 8 a 11, e as instruções ficam na área B, nas colunas 12 a 72, enquanto as colunas 73 a 80 não fazem parte do texto do programa. Neste trabalho, a remoção dos comentários e das linhas de depuração baseou-se na coluna 7 (Seção 4.5).

Figura 1 – Formato fixo de referência de um programa *COBOL*



Fonte: Elaborado pelo autor (2026), com base em IBM (2024) e em linhas do programa *DBCRFUN* (IBM, 2023).

Em *mainframes*, programas como os da *CBSA* são executados sob o CICS (*Customer Information Control System*), subsistema de processamento de transações do sistema operacional z/OS (RAYNS et al., 2011). Esses programas usam comandos embutidos no código: *EXEC CICS*, para os serviços do CICS, e *EXEC SQL*, para o acesso a bancos de dados Db2 (IBM, [2018?]). Após cada comando SQL (*Structured Query Language*), o Db2 informa o resultado no campo *SQLCODE*: zero indica sucesso, +100 indica que nenhum dado foi encontrado e valores negativos indicam erro (IBM, [2016?]b).

2.1.2 Breve histórico

A concepção da linguagem *COBOL* ocorreu em maio de 1959, impulsionada pelo Departamento de Defesa dos Estados Unidos com a meta de estabelecer um padrão universal para aplicações comerciais. Por meio das definições do *Short Range Committee*, a especificação inicial introduziu inovações estruturais, como a rígida separação entre dados e rotinas de execução. Após a sua padronização pelo *American National Standards Institute (ANSI)* em 1968, a linguagem dominou a infraestrutura corporativa global.

O nível crítico de dependência dessa tecnologia tornou-se evidente nas auditorias do problema do milênio. Essas análises identificaram cerca de 800 milhões de linhas de código

COBOL ativas somente no centro financeiro de *Manhattan* (SEBESTA, 2018). Tal cenário ilustra o paradoxo alertado por Bennett (1995). Nesse contexto, sistemas originalmente inovadores transformam-se em âncoras organizacionais. A substituição dessas estruturas apresenta elevada dificuldade devido ao vasto acúmulo histórico de regras de negócio.

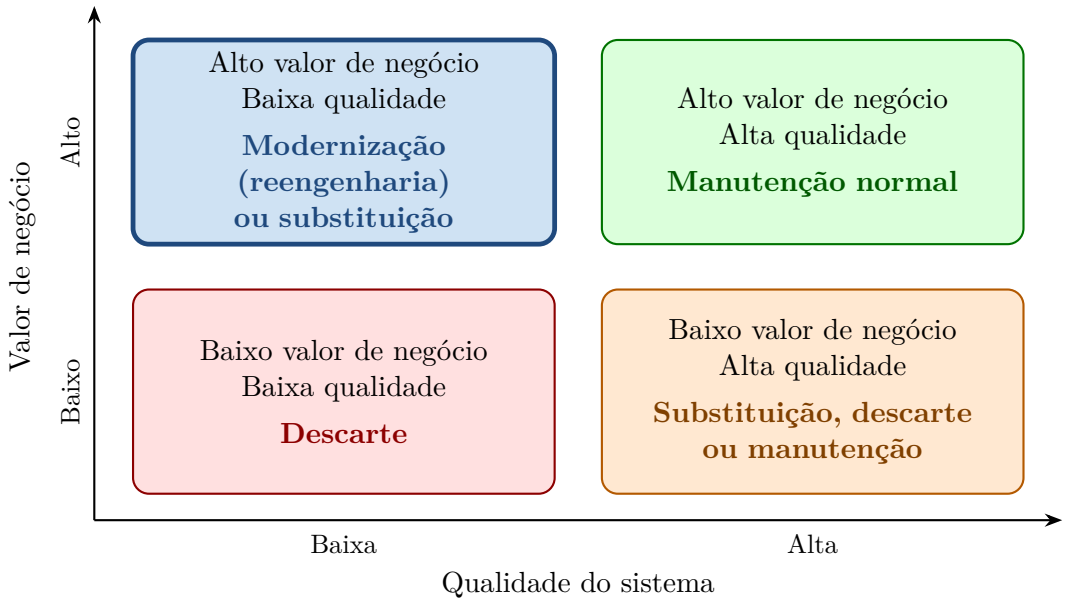
2.1.3 Avanços recentes da área

A Engenharia de *Software* desenvolveu uma base teórica precisa para a análise e a recuperação de sistemas legados. Uma das classificações mais adotadas na área é a taxonomia de Chikofsky e Cross (1990). Estes autores definem a engenharia reversa como a etapa de examinar um sistema para identificar os seus componentes. Esta abordagem possibilita a criação de representações mais abstratas. É importante ressaltar que essa prática consiste apenas na análise estrutural do código, sem o alterar ou replicar. Dessa definição derivam-se processos cruciais para a modernização. O primeiro é a redocumentação, que revisa a estrutura do sistema no mesmo nível de abstração. O segundo é a recuperação de projeto (*design recovery*), que reconstrói a arquitetura ao unir o código-fonte aos conhecimentos de domínio. Esta base teórica viabilizou ferramentas avançadas de *Computer-Aided Software Engineering (CASE)*. Tais recursos permitem que as equipes compreendam arquiteturas críticas de forma independente de manuais obsoletos.

Contudo, a aplicação exclusiva de técnicas manuais de engenharia reversa em bases de código massivas demonstrou um elevado custo financeiro e inviabilidade de tempo. Diante desse obstáculo, a literatura recomenda uma análise estratégica rigorosa antes de qualquer intervenção estrutural (SOMMERVILLE, 2011). A Figura 2 ilustra o modelo de avaliação de sistemas legados, que cruza a qualidade técnica da aplicação com o seu valor de negócio para determinar a abordagem mais adequada: descarte, manutenção, substituição ou modernização.

Na Figura 2, o eixo horizontal representa a qualidade técnica do sistema e o eixo vertical, o seu valor de negócio. Sistemas de baixa qualidade e baixo valor de negócio são candidatos ao descarte, e os de alta qualidade e alto valor permanecem em operação com manutenção normal. Quando a qualidade é alta, mas o valor de negócio é baixo, o sistema pode ser substituído, descartado ou mantido, conforme o custo de mantê-lo. Já os sistemas de alto valor de negócio e baixa qualidade, no quadrante destacado, devem passar por reengenharia ou ser substituídos, se houver um sistema adequado (SOMMERVILLE, 2011).

Figura 2 – Avaliação de sistemas legados



Fonte: Adaptado de Sommerville (2011).

Para esses sistemas, em que a modernização e a redocumentação são imperativas, os avanços recentes da Engenharia de *Software* têm redirecionado o foco da análise manual para a automação baseada em Inteligência Artificial. Estudos como o de Diggs et al. (2025) indicam que *LLMs* podem gerar documentação útil para código legado, embora encontrem mais dificuldade em linguagens de baixo nível, como o *Assembly*. Trabalhos voltados ao *COBOL*, discutidos no Capítulo 3, aplicam esses modelos à extração de regras de negócio (RAJBHOJ et al., 2025; B S; CHIMALAKONDA, 2025). Esta transição tecnológica visa mitigar a dependência de especialistas humanos e acelerar de forma considerável a recuperação de projetos.

2.1.4 Exemplos relevantes

O impacto crítico dos sistemas legados reflete-se na forte dependência estrutural e nos riscos operacionais assumidos por organizações em todo o mundo. No setor financeiro, por exemplo, dados da agência REUTERS (2017) indicam que aplicações baseadas em *COBOL* gerenciam diariamente cerca de três trilhões de dólares em transações bancárias. Esse volume operacional sustenta a base do sistema econômico mundial.

No escopo do setor público, a situação é igualmente alarmante. Um relatório do UNITED STATES. Government Accountability Office (2019) dos Estados Unidos identificou, em 2019, que os sistemas federais mais críticos possuíam até cinco décadas de operação. Essa defasagem consumia a maior parte do orçamento de tecnologia apenas em manutenção. O caso extremo relatado envolveu o Departamento de Defesa americano, cujo *Strategic Automated Command and Control System* coordenava forças nucleares por meio de *hardware* da década de 1970 e disquetes de oito polegadas.

As consequências sociais e diretas dessa estagnação tecnológica afetaram a população de forma severa durante a crise sanitária de 2020, quando o sistema de seguro-desemprego do estado de Nova *Jersey* colapsou. A infraestrutura governamental, sustentada por *mainframes* antigos, não suportou o pico de acessos. Este colapso forçou o governo local a emitir um apelo público à procura de programadores *COBOL* aposentados para restabelecer o pagamento de benefícios essenciais (CNBC, 2020).

2.2 *Large Language Models (LLMs)*

2.2.1 Definições fundamentais

Modelos de Linguagem de Grande Escala (*Large Language Models - LLMs*) definem-se como sistemas computacionais avançados de processamento de linguagem natural, baseados predominantemente em arquiteturas de aprendizado profundo (*deep learning*). O avanço significativo na capacidade desses modelos em compreender a semântica textual foi impulsionado por arquiteturas baseadas em *transformers*. Um marco referencial nessa evolução arquitetural é o modelo *BERT (Bidirectional Encoder Representations from Transformers)*, proposto por Devlin et al. (2019). Esta pesquisa introduziu o treinamento bidirecional profundo. A abordagem viabilizou a análise do contexto de uma palavra por meio da observação simultânea dos termos antecedentes e sucessores. Embora estruturado de forma distinta aos geradores de texto modernos (com foco na estrutura *encoder-only*), o *BERT* estabeleceu a base fundacional para a representação vetorial de palavras.

A categorização de tais sistemas como modelos de "grande escala"(*large*) apoia-se em princípios consolidados na literatura. Kaplan et al. (2020) descreveram as chamadas leis de escalonamento (*scaling laws*). Estes autores comprovaram, por meio de análises experimentais e matemáticas, que o desempenho de redes neurais na modelagem de linguagem melhora de forma contínua e previsível. Esta evolução ocorre quando o sistema é submetido ao aumento de três variáveis estruturais: a quantidade de parâmetros da arquitetura, o volume de dados do conjunto de treinamento (*dataset*) e a capacidade computacional utilizada no processamento (*compute*). Por conseguinte, a eficácia analítica e gerativa dos *LLMs* modernos é resultado direto do escalonamento massivo dessas variáveis. Esta expansão estrutural possibilita a extração de padrões linguísticos e lógicos complexos.

A interação com um *LLM* ocorre por meio de uma instrução em linguagem natural, chamada *prompt*, e a forma dessa instrução influencia a resposta. Na estratégia *zero-shot*, o *prompt* descreve apenas a tarefa; na *few-shot*, inclui também alguns exemplos resolvidos, que o modelo usa como referência sem alteração de seus parâmetros (BROWN et al., 2020). Na estratégia *chain-of-thought*, o *prompt* leva o modelo a explicitar etapas intermediárias de raciocínio antes da resposta final, o que melhora o desempenho em tarefas que exigem várias etapas de inferência (WEI et al., 2022). Essas três estratégias

são comparadas neste trabalho (Seção 4.4).

2.2.2 Breve histórico

A evolução dos *LLMs* ganhou um novo rumo com a transição para arquiteturas puramente generativas. Em contraste com os modelos focados apenas na compreensão de contexto, Radford et al. (2018) propuseram o *GPT* (*Generative Pre-trained Transformer*). Baseado no paradigma *decoder-only*, este estudo demonstrou que o pré-treinamento generativo, seguido de um ajuste para tarefas específicas (*fine-tuning*), permite gerar textos de forma autônoma e coerente.

Um avanço significativo ocorreu com o lançamento do modelo *GPT-3*. Brown et al. (2020) comprovaram que o aumento da rede neural faz emergir a capacidade de aprendizagem em contexto (*in-context learning*). Assim, o modelo consegue resolver problemas lógicos complexos a partir de poucos exemplos fornecidos na própria instrução (*few-shot learners*). Essa característica dispensa a necessidade de reajustes constantes na estrutura interna do sistema.

O amadurecimento desta tecnologia resultou na sua aplicação prática generalizada. A criação de interfaces conversacionais otimizadas (OPENAI, 2022) demonstrou a utilidade dos *LLMs* em múltiplos cenários operacionais. Posteriormente, a apresentação do GPT-4 (OPENAI, 2023) estabeleceu um novo estado da arte (*state-of-the-art*), assegurando maior precisão e raciocínio lógico. Tais capacidades tornam esses sistemas aptos para desafios de alta complexidade analítica, a exemplo da engenharia reversa e da interpretação de código em sistemas legados.

2.2.3 Avanços recentes da área

Os desenvolvimentos mais recentes na literatura sobre *LLMs* focam na otimização e no alinhamento. A revisão sistemática de Minaee et al. (2024) evidencia que a evolução da área deixou de depender apenas do aumento massivo das redes. O foco expandiu-se para abordagens complementares, como a Geração Aumentada por Recuperação (*Retrieval-Augmented Generation - RAG*), a criação de agentes autônomos e o alinhamento comportamental. Em particular, a introdução do alinhamento por *feedback* humano de Ouyang et al. (2022) assegura o cumprimento de instruções. Esse alinhamento constitui um requisito indispensável para aplicar a inteligência artificial na engenharia de *software*.

Paralelamente, o domínio isolado das tecnologias de código fechado (*closed-source*) diminuiu com a ascensão de modelos eficientes em código aberto (*open-source*). A proposta da arquitetura *LLaMA* por Touvron et al. (2023) comprovou a possibilidade de atingir alta capacidade analítica com menor exigência de infraestrutura computacional. Essa eficiência viabiliza a execução de sistemas avançados diretamente em ambientes locais (*on-premise*). Tal característica permite a modernização prática de sistemas legados

bancários, uma vez que o código-fonte contém regras de negócio sigilosas que não podem transitar por servidores de nuvem externos.

2.2.4 Exemplos relevantes

A viabilidade prática das capacidades analíticas dos *LLMs* na engenharia de *software* é evidenciada pelo desenvolvimento de modelos treinados de forma específica para compreender linguagens de programação. Um marco ilustrativo dessa especialização é o modelo *Codex*, apresentado por Chen et al. (2021). Este sistema comprovou que redes neurais ajustadas (*fine-tuned*) com vastos volumes de código-fonte extraídos de repositórios públicos conseguem interpretar a sintaxe estruturada e gerar rotinas computacionais com alta precisão.

A consolidação dessa área impulsionou a criação de alternativas de código aberto (*open-source*). Projetos como o *CodeGen* (NIJKAMP et al., 2022) e o *StarCoder* (LI et al., 2023) provaram que arquiteturas acessíveis também são eficientes para analisar e gerar algoritmos. Diante do desafio de manter sistemas antigos, a existência dessas ferramentas indica que a inteligência artificial pode apoiar a engenharia reversa. Desse modo, esses modelos formam a base ideal para automatizar a extração de regras de negócio em linguagens legadas, como o *COBOL*.

Capítulo 3

Trabalhos Relacionados

Os trabalhos relacionados foram identificados por buscas no Google Scholar e nas bibliotecas digitais IEEE Xplore e ACM Digital Library, com combinações de termos como *COBOL*, *legacy code*, *large language model* (LLM), *business rules*, *documentation* e *code comprehension*, sem restrição de data. Foram considerados relacionados os estudos que aplicam ou avaliam LLMs em código legado com ao menos um destes objetivos: extrair regras de negócio, gerar documentação ou medir a compreensão do código pelos modelos. Entre os resultados, priorizaram-se os estudos que foram mais citados e com datas mais recentes, o que concentrou a seleção em estudos de 2025 e 2026. A Seção 3.1 reúne os estudos sobre extração de regras de negócio em *COBOL*, a Seção 3.2 trata da documentação e da compreensão de código legado, e a Seção 3.3 compara os trabalhos entre si e com esta pesquisa.

3.1 Extração de regras de negócio em *COBOL*

No contexto da modernização de sistemas legados, Rajbhoj et al. (2025) têm como objetivo principal extrair a lógica de negócios de módulos *COBOL* industriais por meio de modelos de linguagem de grande escala (*LLMs*). Para isso, o método proposto adota uma arquitetura do tipo *Maker-Checker*, com sólida base em técnicas de engenharia de *prompt* para analisar o código legado. Os principais resultados demonstram que a abordagem alcançou uma precisão média de 91%. Contudo, o método apresentou limitações e falhas na interpretação de variáveis dinâmicas e da instrução *GO TO* (comando de desvio incondicional). Este estudo alinha-se a este trabalho, pois valida a viabilidade do uso de *LLMs* para a extração de regras em *COBOL*. Adicionalmente, os indicadores alcançados servem como linha de base (*baseline*) para a comparação com os resultados desta pesquisa. Destaca-se a relevância da referida publicação, aceita no *Automated Software Engineering Workshop (ASEW)*. Trata-se de um evento internacional vinculado à conferência *ASE*, de reconhecida tradição em Engenharia de *Software* e promovido de forma conjunta pelas instituições *IEEE* e *ACM*. Esse cenário indica a relevância dos resultados apresentados.

Com foco na extração e na sumarização de regras de negócio, o estudo de B S e Chimalakonda (2025) apresenta a ferramenta *COBRAIN*. Trata-se de um recurso baseado em *few-shot prompting* aplicado a código *COBOL* legado. Para validar a proposta, os autores realizaram uma avaliação comparativa em relação ao *COBREX*, uma abordagem tradicional baseada em regras algorítmicas. Tendo como referência a saída do *COBREX*, a ferramenta *COBRAIN* alcançou precisão de 1,0, mas obteve revocação (*recall*) inferior (0,746). Na prática, esse resultado indica que a inteligência artificial apresenta elevada exatidão no conteúdo extraído, embora omita lógicas que o método tradicional consegue capturar. No entanto, ao avaliar ambas as abordagens perante um conjunto de dados de referência (*ground truth*), a ferramenta *COBRAIN* obteve desempenho superior, com a métrica *F1 score* de 0,73. Esse valor superou a marca de 0,59 alcançada pelo *COBREX*. Além da análise quantitativa, um teste prático revelou que mais de 80% dos 28 participantes preferiram as regras geradas pelos modelos de linguagem. Esta pesquisa contribui para o presente trabalho por dois motivos centrais: consolida o uso de métricas quantitativas e evidencia o *trade-off* entre precisão e revocação, aspecto também medido nesta pesquisa. A relevância científica do estudo é atestada pela publicação na *International Conference on Evaluation and Assessment in Software Engineering (EASE)*, evento promovido de forma conjunta pelas instituições *ACM* e *IEEE*.

No contexto de construção de arquiteturas integradas, Kalohia (2026) propõe um *pipeline* híbrido para extrair requisitos de negócio a partir de sistemas em *COBOL* e *PL/I*. A metodologia une a análise tradicional de código, como o *parsing* e o mapeamento de fluxo de dados, à inteligência artificial generativa. Quando aplicada a um cenário real da indústria financeira com mais de 3,4 milhões de linhas de código, a ferramenta produziu especificações precisas e com total rastreabilidade. Em termos quantitativos, o modelo atingiu 93% de concordância com as regras elaboradas por especialistas humanos. Adicionalmente, observou-se uma redução de cerca de 70% no esforço de documentação e uma aceleração de 3,2 vezes no tempo de análise. Este artigo é uma referência para a extração de regras de negócio em larga escala e para a comparação com especialistas humanos, embora a métrica reportada, de concordância, seja diferente da precisão usada neste trabalho. A relevância desta investigação é atestada pela publicação na *Association for Computational Linguistics (ACL)*, por meio da *Industry Track*, um fórum de reconhecido prestígio em processamento de linguagem natural.

3.2 Documentação e compreensão de código legado

No que diz respeito à redocumentação automatizada, Diggs et al. (2025) avaliaram a viabilidade do uso de *LLMs* para gerar documentação em código legado. A metodologia utilizou estratégias de *prompting* na criação de comentários explicativos em nível de linha. Estes testes foram aplicados em dois conjuntos de dados distintos: um em *MUMPS*, com

foco em sistemas de prontuários eletrônicos, e outro em *Assembly mainframe (ALC)*. Além disso, os autores adotaram uma rubrica rigorosa para medir a completude, a legibilidade, a utilidade e a ocorrência de alucinações. Os resultados demonstram que a documentação gerada apresentou alta precisão e utilidade no caso do *MUMPS*. Em contrapartida, linguagens de mais baixo nível, como o *Assembly*, apresentaram maiores desafios de interpretação para os modelos. Esta investigação contribui para o trabalho, uma vez que fornece a fundamentação teórica e as métricas de avaliação necessárias para estruturar a documentação de código *COBOL* com mitigação de riscos. A relevância e a confiabilidade científica do artigo são atestadas pela publicação no *International Workshop on Large Language Models for Code (LLM4Code)*. Este evento é vinculado à conferência *International Conference on Software Engineering (ICSE)*, promovida pelas instituições *IEEE* e *ACM*.

No âmbito da avaliação de modelos de linguagem, Sabetto et al. (2025) investigam o impacto da documentação na interpretação de sistemas legados. Para isso, os autores aplicaram questões de múltipla escolha (*multiple-choice question answering - MCQA*). O objetivo consistiu em mensurar o entendimento dos modelos sobre a sintaxe e a semântica do código. Os resultados revelam um comportamento peculiar: comentários precisos elevam a exatidão das respostas e inconsistências pontuais são quase ignoradas. No entanto, perante uma documentação com incorreções profundas, os *LLMs* assumem as informações falsas como verdadeiras. Isso provoca uma quebra acentuada na precisão interpretativa. Esse achado é relevante para o trabalho, pois indica que os modelos podem confiar em excesso nos textos explicativos, o que reforça a decisão de construir o gabarito a partir do código e de retirar os comentários da entrada enviada aos modelos. Embora se trate de um relatório técnico, a relevância e o nível de impacto dos achados fundamentam-se no prestígio da *MITRE Corporation*, organização de referência na área de pesquisa e desenvolvimento tecnológico.

3.3 Comparação entre os trabalhos

A Tabela 1 resume os trabalhos discutidos, com a linguagem ou o contexto, a abordagem, as principais métricas e o veículo de cada um. Entre eles, apenas Rajbhoj et al. (2025) e B S e Chimalakonda (2025) reportam métricas diretamente comparáveis às deste trabalho, o que orienta a análise comparativa.

Este trabalho aproxima-se desses dois estudos pelo objeto, as regras de negócio em código *COBOL*, e pela métrica de precisão. Além disso, reúne três elementos: um gabarito construído a partir de uma aplicação de código aberto, com cada regra conferida no código-fonte; a comparação entre três estratégias de *prompt* em dois modelos de fornecedores distintos; e a classificação de cada item gerado em alucinação, enriquecimento ou reformulação, que evita contar como erro uma regra correta ausente do gabarito.

Tabela 1 – Comparativo dos trabalhos relacionados

Trabalho	**Linguagem/ Contexto**	**Abordagem/ Método**	**Principais métricas/ resultados**	**Veículo**
Rajbhoj et al. (2025)	*COBOL* industrial	Arquitetura *Maker-Checker* com *prompting*	91% de precisão média (dificuldades em *GO TO*)	*ASEW (IEEE/ACM)*
B S e Chimalakonda (2025)	*COBOL* legado	*Few-shot prompting* vs. regras (*COBREX*)	F1 = 0,73; 1,0 de precisão (vs. *COBREX*); 80% de aprovação dos usuários	*EASE (ACM/IEEE)*
Kalohia (2026)	*COBOL* e *PL/I* (3,4M linhas)	*Pipeline* híbrido (*parsing* + *LLM*)	93% de concordância; 70% menos esforço; 3,2x mais rápido	*ACL Industry Track*
Diggs et al. (2025)	*MUMPS* e *Assembly (ALC)*	Comentários linha a linha com rubrica de avaliação	Documentação útil e precisa; maiores desafios em *Assembly*	*LLM4Code / ICSE (IEEE/ACM)*
Sabetto et al. (2025)	Código legado geral	Avaliação objetiva via *MCQA*	Documentação muito incorreta induz alucinações no *LLM*	Relatório técnico (*MITRE*)

Fonte: Elaborado pelo autor (2026), com base nos trabalhos citados.

Capítulo 4

Método

Para avaliar a documentação gerada por *LLMs* a partir de código *COBOL* legado, este trabalho propõe uma metodologia multicritério focada em regras de negócio atômicas. Essa abordagem busca contornar uma limitação das métricas tradicionais de comparação textual. Em geral, essas métricas penalizam os modelos generativos quando eles apenas reformulam ou enriquecem o conteúdo original (ROY; FAKHOURY; ARNAOUDOVA, 2021).

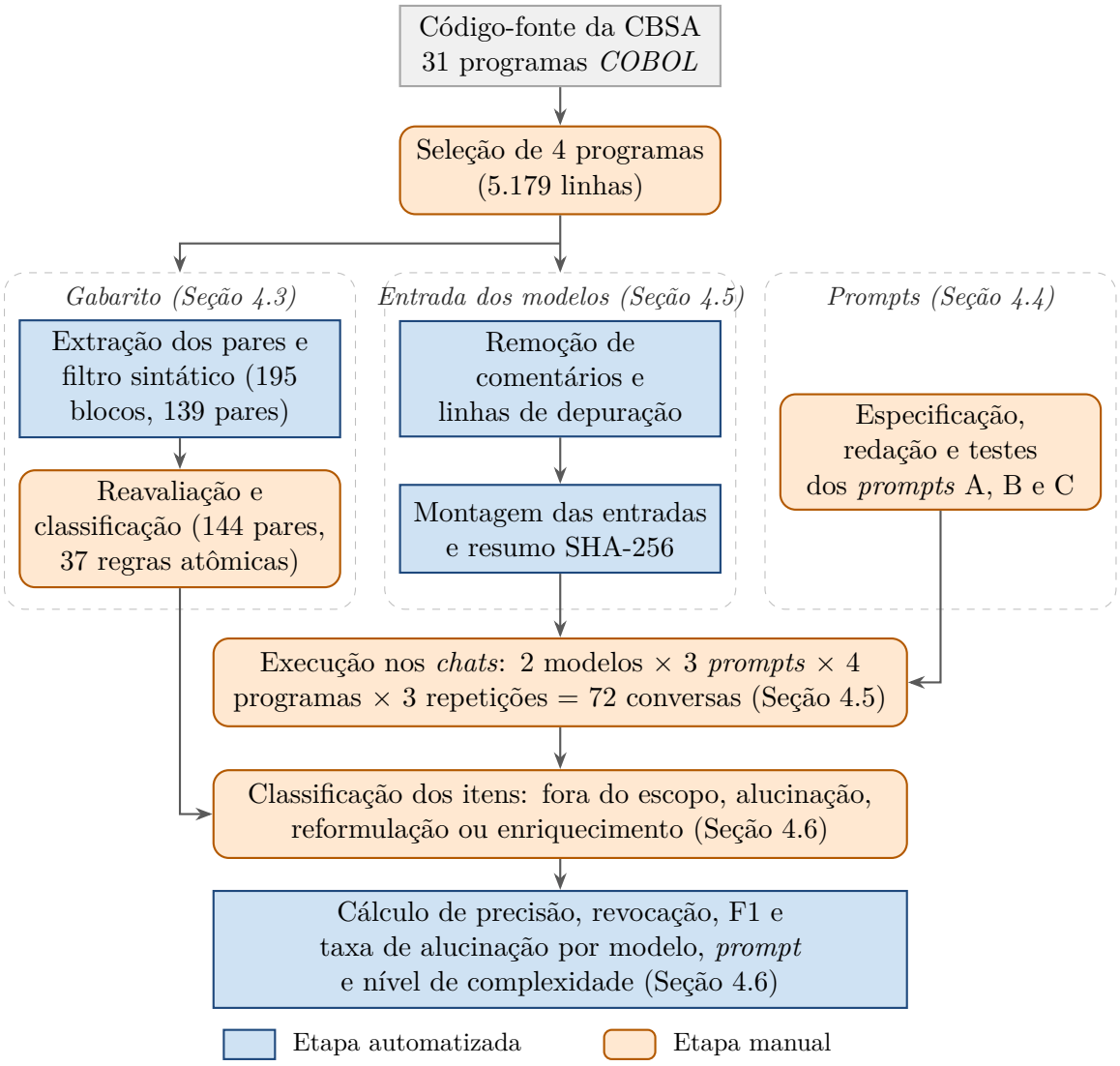
O escopo restringe-se às regras de negócio. Esse recorte metodológico alinha-se aos trabalhos de referência (RAJBHOJ et al., 2025; B S; CHIMALAKONDA, 2025; KALOHIA, 2026). A inclusão de outros tipos de documentação, como arquitetura e modelo de dados, exigiria gabaritos e métricas próprios. Além disso, essa ampliação inviabilizaria a comparação direta com as publicações citadas.

Do ponto de vista metodológico, o estudo classifica-se como uma pesquisa de natureza científica e aplicada. A investigação busca responder às questões apresentadas na Seção 1.3 por meio de experimentação e análise de resultados. Trata-se de uma avaliação controlada, com entradas, *prompts* e critérios fixados antes da execução.

4.1 Visão geral

A Figura 3 apresenta uma visão geral do procedimento de avaliação. Após a seleção dos quatro programas da *CBSA*, três frentes foram preparadas de forma independente: o gabarito e as entradas enviadas aos modelos, construídos a partir desses programas, e os *prompts*, testados em programas fora da amostra. As entradas e os *prompts* compuseram as 72 conversas com os modelos avaliados, e cada item das respostas foi classificado por comparação com o gabarito, o que permitiu o cálculo das métricas. As etapas automatizadas foram executadas por *scripts* em *Python* ou pela planilha do experimento, e as demais dependeram de decisões e ações do discente.

Figura 3 – Visão geral do procedimento de avaliação



Fonte: Elaborado pelo autor (2026).

O desenvolvimento do trabalho resultou em três produtos principais:

- ❏ Procedimento de geração de documentação de regras de negócio a partir de código *COBOL*, composto por *scripts* de preparação da entrada, publicados no repositório da pesquisa, e por um protocolo de execução nos modelos *Claude Sonnet* 5.5 e *GPT-6* Sol.
- ❏ Gabarito (*ground truth*) estruturado, composto por regras de negócio extraídas do sistema de referência e ligadas aos trechos de código-fonte que as implementam.
- ❏ Relatório comparativo de métricas de qualidade da documentação gerada, que avalia os resultados em relação aos valores de base (*baselines*) reportados na literatura.

Os modelos avaliados foram o *Claude Sonnet 5.5*, da *Anthropic* (ANTHROPIC, 2026), e o *GPT-6* Sol, da *OpenAI* (OPENAI, 2026b). O *Claude Sonnet 5.5*, versão oferecida

pelo plano Claude Pro, sucede o Claude Sonnet 5, que apresentou desempenho superior na avaliação de estratégias arquiteturais e na fundamentação de decisões (GRAHL, 2026). O GPT-6 Sol corresponde à opção "GPT-6" do ChatGPT Plus, equivalência que consta na central de ajuda da OpenAI, consultada em 17/10/2026 (OPENAI, 2026a). O GPT-5.5, previsto na etapa inicial, foi retirado do ChatGPT em 14/10/2026. A escolha de modelos de fornecedores distintos permitiu comparar famílias diferentes na mesma tarefa.

As execuções ocorreram por meio da interface de conversação (*chat*) dos dois serviços, com o nível intermediário de raciocínio (*Medium*) em ambos. O uso das interfaces de programação de aplicações (*APIs*)[1] foi descartado devido ao custo. Como a interface de conversação não permite fixar a temperatura de geração, cada combinação de programa, *prompt* e modelo foi executada três vezes, em conversas novas, e os resultados reportados consideram a variação entre essas repetições.

A linguagem *Python* (3.14.2)[2] foi empregada nos *scripts* de preparação dos dados. As tarefas abrangeram o inventário dos programas da *CBSA*, a extração dos pares de comentário e código e a geração do gabarito. Também incluíram a remoção dos comentários e das linhas de depuração, validada por um teste de reconstrução, e a montagem da entrada de cada execução, com o registro do seu resumo criptográfico. As planilhas do gabarito e do experimento foram geradas com a biblioteca *openpyxl*[3], e todos os *scripts* estão publicados no repositório da base de dados desta pesquisa (`math-mcruz/SOURCE-TCC`)[4].

4.2 Base de dados

Como sistema de referência para a investigação, adotou-se a *CICS Banking Sample Application* (*CBSA*), aplicação bancária de exemplo disponibilizada pela *IBM* sob a licença *Eclipse Public License 2.0* (IBM, 2023). O sistema é composto por 31 programas *COBOL*, que somam cerca de 26 mil linhas, sem contabilizar os *copybooks*. A aplicação conta com documentação estruturada em diferentes níveis, que abrange desde a arquitetura e guias do usuário até comentários internos no código-fonte.

Para a realização do estudo, utilizou-se o diretório `src/base` do repositório `ijmitch/cics-bankin` no *commit* `e224756`[5], datado de 27/09/2023, visto que o repositório oficial deixou de disponibilizar o código-fonte em julho de 2026. Uma cópia fiel do código foi preservada no repositório desta pesquisa (`math-mcruz/SOURCE-TCC`). A seleção desse sistema fundamentou-se em três pilares:

❑ Realismo: a *CBSA* é um sistema de porte real, caracterizado pelo estilo de có-

[1] Documentação das APIs: <https://platform.openai.com/docs> (OpenAI) e <https://docs.claude.com> (Anthropic). Acesso em: 9 nov. 2026.
[2] Disponível em: <https://www.python.org>. Acesso em: 9 nov. 2026.
[3] Disponível em: <https://openpyxl.readthedocs.io>. Acesso em: 9 nov. 2026.
[4] Disponível em: <https://github.com/math-mcruz/SOURCE-TCC>. Acesso em: 9 nov. 2026.
[5] Identificador completo: e22475691524ef72a92ee77f7040233e098ab1b9.

digo corporativo e por operações bancárias típicas, a exemplo de débito, crédito e transferência.

- ❏ Validação: os comentários internos explicam os blocos lógicos do código e serviram como ponto de partida para a construção do gabarito, com a conferência individual de cada regra no próprio código-fonte. Dos 29 comentários empregados no gabarito, 9 apresentavam-se defasados ou incompletos frente ao código (Seção 4.3).

- ❏ Acessibilidade técnica: o código pode ser analisado de forma estática. Programas *CICS* com acesso ao *Db2* exigem o tradutor de comandos do *CICS* e o pré-compilador do *Db2* para a sua compilação (IBM, [2018?]). Como essa infraestrutura não se encontrava disponível, os programas não foram compilados nem executados.

O uso de sistemas *COBOL* reais em avaliações de documentação também foi adotado por Lei et al. (2025), que analisaram explicações de código geradas por *LLMs* em 14 projetos *COBOL* de código aberto. O referido trabalho é um *preprint*, sem publicação em veículo revisado por pares até a consulta efetuada em 08/11/2026. Ainda assim, o estudo relata a avaliação manual das explicações por dois anotadores, com concordância medida pelo coeficiente kappa de Cohen, o que reforça a viabilidade da abordagem.

Em razão do volume total do sistema, a avaliação concentrou-se em quatro programas. A seleção obedeceu a três critérios: a implementação das operações centrais de débito, crédito e transferência; a compatibilidade do volume de código com a revisão manual do gabarito no prazo disponível; e a presença de diversidade em estruturas de decisão, como validações, tratamento de exceções e regras condicionais. Cada operação é implementada em duas camadas: um programa de tela, que valida os dados inseridos pelo usuário e exibe mensagens, e um programa de processamento, acionado pela tela para aplicar a operação na conta e retornar os códigos de falha.

A Tabela 2 detalha os programas selecionados, que somam 5.179 linhas e correspondem a cerca de 20% do total de linhas dos programas da *CBSA*. Os modelos receberam o código desses programas sem os comentários e sem as linhas de depuração, conforme descrito na Seção 4.5.

Tabela 2 – Programas selecionados para a avaliação

Programa	Operação	Camada	Linhas	Regras no gabarito
DBCRFUN	Débito e crédito	Processamento	861	9
BNK1CRA	Débito e crédito	Tela	1.168	8
XFRFUN	Transferência	Processamento	1.924	8
BNK1TFN	Transferência	Tela	1.226	12
Total			5.179	37

Fonte: Elaborado pelo autor (2026).

4.3 Construção do gabarito

A análise inicial do código-fonte da *CBSA* revelou comentários integrados aos blocos lógicos, em vez de restritos aos cabeçalhos dos programas. O gabarito (*ground truth*) foi construído a partir de pares formados por um bloco de comentário e o código que o segue, até o próximo comentário, com limite de 50 linhas por trecho. Um *script* extraiu esses pares da *PROCEDURE DIVISION* dos quatro programas selecionados. Com isso, os comentários de cabeçalho ficaram de fora, e foram obtidos 195 blocos.

Um filtro sintático reteve os pares com estruturas de decisão (*IF, EVALUATE* ou *WHEN*) ou de tratamento de falha, como o término anormal da tarefa (*ABEND*), o teste do código de retorno do *Db2* (*SQLCODE*) e os testes de resposta dos comandos do *CICS*. Foram retidos 139 pares, e o restante foi registrado como descartado para assegurar a auditabilidade do critério. Como o filtro automático não detecta condições implícitas no fluxo, os trechos descartados cujo comentário mencionava uma condição foram reavaliados por meio de inspeção manual. Cinco pares foram reincluídos nessa etapa, a exemplo da atualização de saldo do programa *DBCRFUN*. Nesses casos, classificados como regras procedurais, a execução do trecho depende da satisfação de testes anteriores.

A revisão manual corrigiu falhas pontuais da extração automática. Os ajustes abrangeram comentários adjacentes fundidos em um único bloco, separações incorretas entre condição e consequência, e trechos que ultrapassavam o limite de uma *SECTION*. Ao final dessa triagem, obteve-se um total de 144 pares candidatos. A classificação seguiu critérios definidos em conjunto com o orientador em 16/10/2026. O critério específico de campo obrigatório, detalhado a seguir, foi aprovado em 02/11/2026.

Considerou-se regra de negócio a lógica reconhecida pela área de negócio, representada por perfis como gerentes ou clientes do banco, em oposição a detalhes de implementação. Três categorias de pares foram excluídas do escopo. A primeira, infraestrutura, agrupou o tratamento de erro padrão, comum a qualquer programa *COBOL*, e as falhas técnicas do *Db2* (*SQLCODE* diferente de 0 e de +100). Essa classe também abarcou o desfazimento de transações (*rollback*), o *ABEND*, o mecanismo *storm drain* da *CBSA*, que trata a perda de conexão com o *Db2*, a gravação na tabela de transações processadas (*PROCTRAN*), a respectiva sinalização de sucesso e a ordem de atualização das contas. A segunda categoria, interface, abrangeu as teclas de função, a navegação de telas e a validação de formato dos dados de entrada. Esta última exclusão resultou de uma extensão de critério proposta pelo discente. A terceira categoria excluiu lógicas desprovidas de comentário próprio e que se encontravam contidas no interior de outro par.

Apesar do filtro de interface, algumas checagens de tela permaneceram classificadas como regra de negócio, a exemplo da obrigatoriedade de preenchimento e das restrições de valor de um campo. Nos casos de divergência entre o teste condicional e a mensagem exibida, adotou-se o comportamento efetivo do código como referência. O resultado de uma consulta ao banco de dados pode originar uma regra, como na ausência de registro

(*SQLCODE* igual a +100). No entanto, as operações de acesso a dados não foram tratadas como regras isoladas. Se a decisão depende de uma busca, a regra documenta a consequência para o negócio, e não o comando técnico acionado. Situações com incerteza sobre o fluxo de execução foram excluídas e registradas.

Cada item do gabarito é composto por uma condição e uma consequência. Pares com lógicas múltiplas foram desdobrados em regras atômicas. A identificação ocorreu pelo código do par seguido de um sufixo numérico, a exemplo de *DBCRFUN*-008.1 e *DBCRFUN*-008.2. Ocorrências repetidas da mesma regra no código geraram entradas independentes. O gabarito descreve o comportamento real do programa. Quando a anotação original divergia do código, a inconsistência era registrada. Cada par recebeu uma marcação sobre a qualidade do comentário de origem: defasado (em contradição com o código), incompleto (coerente, porém parcial) ou correto.

A Figura 4 ilustra o processo com um par do programa *DBCRFUN*, retido pelo filtro sintático. O código distingue dois casos de falha na consulta da conta. A ausência da conta (*SQLCODE* igual a +100) originou a regra *DBCRFUN*-005, pois o código de falha '1' tem significado para a área de negócio, enquanto as demais falhas da consulta (código '2') foram classificadas como infraestrutura.

Figura 4 – Exemplo de par do gabarito e da regra atômica correspondente

(a) Par de comentário e código do programa *DBCRFUN* (linhas 276 a 287)

```
276        *    Check that select was successful. If it wasn't then deal with
277        *    it
278        *
279             IF SQLCODE NOT = 0
280
281                MOVE 'N' TO COMM-SUCCESS
282
283                IF SQLCODE = +100
284                   MOVE '1' TO COMM-FAIL-CODE
285                ELSE
286                   MOVE '2' TO COMM-FAIL-CODE
287                END-IF
```

⇓ *classificação como regra de negócio*

(b) Regra no gabarito (*DBCRFUN*-005)

Se a conta solicitada não for encontrada (*SQLCODE* igual a +100), a operação é sinalizada como malsucedida com o código de falha '1'.

Fora da regra: o ramo *ELSE* (código de falha '2'), falha técnica do *Db2* classificada como infraestrutura.

Fonte: Elaborado pelo autor (2026), com código de IBM (2023).

A Tabela 3 resume as etapas do processo. Do total de 144 pares candidatos, 29 foram validados como regra de negócio e desdobrados em 37 regras atômicas. Esse conjunto forma a versão V11 do gabarito e constitui a unidade base para o cálculo das métricas.

Nos 29 pares retidos, o comentário original apresentou-se defasado em 4 casos, incompleto em 5 e correto em 20.

Tabela 3 – Extração e classificação dos pares do gabarito

Etapa	Pares
Blocos de comentário extraídos	195
Retidos pelo filtro sintático	139
Reincluídos na reavaliação manual	5
Pares candidatos	144
Excluídos como infraestrutura	93
Excluídos como interface	18
Excluídos por não ter comentário próprio	4
Regra de negócio (37 regras atômicas)	29

Fonte: Elaborado pelo autor (2026).

O discente inspecionou cada par no próprio código-fonte. O orientador, por sua vez, participou da definição dos critérios de classificação e arbitrou os casos de dúvida ao longo de reuniões semanais. A redação preliminar das regras e de suas respectivas categorias contou com o apoio de um modelo de linguagem. O resultado dessa etapa foi conferido de forma integral pelo discente (Seção 4.8). Diferentemente da abordagem adotada por Lei et al. (2025), a concordância entre avaliadores não foi quantificada estatisticamente, fator abordado na discussão sobre as limitações do estudo.

4.4 Estratégias de *prompt*

O estudo comparou três estratégias de *prompt*:

- Estratégia A (*zero-shot*): o modelo recebeu apenas a instrução e o código (BROWN et al., 2020).
- Estratégia B (*few-shot*): o modelo recebeu também dois exemplos resolvidos (BROWN et al., 2020). Essa é também a técnica adotada pela ferramenta *COBRAIN* para código *COBOL* (B S; CHIMALAKONDA, 2025).
- Estratégia C (*chain-of-thought*): o *prompt* solicitou a escrita de uma seção de análise antes da lista de regras (WEI et al., 2022).

Os três *prompts* compartilham a mesma base, com variação restrita ao bloco da estratégia. Essa base define a tarefa e o conceito de regra de negócio. O texto lista os elementos excluídos do escopo, como erro padrão, falhas técnicas, registros de auditoria, navegação e formato de campos. A instrução também indica o que deve permanecer, a exemplo da obrigatoriedade e das restrições sobre o valor em checagens de campo. Além disso, a base restringe a resposta às lógicas ligadas a decisões no código, orienta o agrupamento de

valores com a mesma consequência e exige uma regra por item, composta por condição e consequência. O mesmo texto em português foi submetido aos dois modelos.

O discente elaborou a especificação dos *prompts* com o apoio do modelo *Claude Opus 5.5*. A redação final do texto coube ao modelo *Gemini 3.1 Pro*, usado por meio do *Gemini Notebook* (antigo *NotebookLM*)[6]. A escolha desse modelo não avaliado garantiu que os modelos avaliados não participassem da montagem de seus próprios *prompts*. Foram produzidas quatro versões, e os testes usaram apenas dois programas fora da amostra principal, *CREACC* e *BNK1CAC*. A primeira versão foi descartada sem teste por conter exemplos coincidentes com as regras do gabarito. A segunda versão produziu listas de 24 a 33 itens, com a inclusão de atribuições de valores e de campos exibidos. A terceira versão restringiu o escopo às decisões, mas apresentou granularidade instável. A quarta versão introduziu orientações de agrupamento e cumpriu o formato exigido nos 12 testes realizados, com a geração de listas de 4 a 17 itens. Os *prompts* foram congelados nessa versão em 24/10/2026 e mantidos sem alterações posteriores.

Durante os testes da terceira versão, a interface do *Claude* bloqueou o *prompt* C com um aviso de segurança. O sistema interpretou o pedido para "raciocinar por etapas" como uma solicitação de acesso ao raciocínio interno do modelo. Na versão final, o *prompt* passou a solicitar a análise como uma seção escrita da resposta. Essa adaptação preserva a característica central da estratégia, baseada na geração de passos intermediários escritos antes da conclusão (WEI et al., 2022).

Os exemplos do *prompt* B foram extraídos de programas externos à amostra avaliada: *UPDCUST* (linhas 136 a 244) e *BNK1UAC* (linhas 491 a 534). As respostas esperadas foram rascunhadas com o apoio do *Claude Opus 5.5* e validadas pelo discente. Um dos exemplos obedece a um padrão presente em três regras do gabarito (*DBCRFUN*-005, *XFRFUN*-034 e *XFRFUN*-043), nas quais um registro não encontrado resulta em um código de falha. Essa semelhança tem o potencial de favorecer o acerto dessas regras específicas nesse *prompt*. O exemplo permaneceu na configuração final com a aprovação do orientador, e o efeito dessa decisão é analisado na seção de resultados. O texto completo dos três *prompts* encontra-se no Apêndice A.

4.5 Preparação da entrada e execução

Visto que os comentários internos integram a documentação original da *CBSA* e constituem o ponto de partida do gabarito, tais elementos foram removidos do código-fonte submetido aos modelos. A remoção ocorreu por meio de um *script* em *Python*, o qual classifica cada linha a partir do caractere situado na coluna 7, correspondente à área indicadora do formato fixo do *COBOL*. Nessa estrutura, o asterisco sinaliza comentário, enquanto as letras D e d indicam linhas de depuração. Essas linhas de depuração

[6] Disponível em: <https://notebook.google>. Acesso em: 10 nov. 2026.

também foram eliminadas, uma vez que nenhum dos quatro programas ativa o modo correspondente (cláusula *WITH DEBUGGING MODE*), hipótese na qual o compilador as interpreta como comentários (IBM, [2016?]a). A exclusão abrangeu quatro linhas no total, divididas igualmente entre os programas *DBCRFUN* e *BNK1TFN*, sem que qualquer regra do gabarito dependesse desses trechos.

Em razão da indisponibilidade de compilação (Seção 4.2), a validação da remoção ocorreu em duas camadas. A primeira etapa, automatizada, verificou a ausência de tabulações, de caracteres externos ao padrão ASCII (*American Standard Code for Information Interchange*), de conteúdo nas colunas 73 a 80 e de comentários em linha, uma vez que tais ocorrências comprometeriam a classificação baseada na coluna indicadora. Na sequência, um teste de reconstrução intercalou os arquivos de saída na ordem original e comparou o resultado com o programa fonte, linha por linha, com resultado idêntico nos quatro programas. A segunda camada consistiu em uma verificação manual por amostragem para atestar se a regra da coluna 7 separava adequadamente comentários e código executável, aspecto não avaliado pelo teste de reconstrução por aplicar a mesma lógica em ambas as direções. A Tabela 4 detalha o quantitativo de linhas processadas.

Tabela 4 – Linhas removidas do código enviado aos modelos

Programa	Linhas no original	Código	Comentário	Depuração
DBCRFUN	861	665	194	2
XFRFUN	1.924	1.582	342	0
BNK1CRA	1.168	993	175	0
BNK1TFN	1.226	1.030	194	2
Total	5.179	4.270	905	4

Fonte: Elaborado pelo autor (2026).

As mensagens exibidas ao usuário foram mantidas no código-fonte, uma vez que integram o comportamento do programa e constituem consequências de regras presentes no gabarito. Os comandos *COPY* não foram expandidos, o que impediu o envio das definições de dados contidas nos *copybooks* (arquivos incorporados por essa instrução), embora a identificação dos campos permaneça visível no próprio código. Ambos os pontos são analisados na seção de limitações do trabalho.

A construção da entrada de cada execução ocorreu por meio de *script*, com a inserção do código no lugar do marcador reservado no *prompt* (Apêndice A), sem modificações adicionais além da eliminação de linhas em branco ao final do arquivo. Para cada arquivo de entrada, foi gerado o resumo criptográfico *SHA*-256 (*Secure Hash Algorithm*) (NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY, 2015), um identificador de 64 caracteres sensível a qualquer alteração no conteúdo. Esse registro permite verificar que o texto submetido aos modelos corresponde exatamente ao arquivo produzido e que os dois modelos receberam entradas idênticas em cada combinação.

As execuções oficiais contabilizaram 72 conversas, resultantes da combinação entre quatro programas, três *prompts*, dois modelos e três repetições. Cada execução foi conduzida em uma conversa nova e independente, por meio do *Chat* Temporário no *ChatGPT* e do *chat* anônimo no *Claude*. Essas sessões ocorreram fora de projetos, sem memória, sem personalização e sem busca na *web*, com o *prompt* e o código enviados juntos em uma única mensagem. O procedimento organizou-se por rodadas, com a conclusão de todas as combinações da primeira repetição antes do início da segunda. Toda resposta incompleta, com uso de busca na *web*, com alteração no modelo exibido ou com erro de interface foi descartada e refeita em uma nova conversa.

Em cada execução, foram registrados a data, o horário, o modelo informado pela plataforma, o nível de raciocínio, a versão do *prompt*, o resumo *SHA*-256 da entrada e o arquivo com a resposta, copiada pela própria interface e armazenada sem modificações, além da captura de tela da sessão. Como as conversas temporárias não permanecem salvas no histórico dos serviços, esses arquivos constituem o registro das execuções. Antes do experimento oficial, um teste piloto com 12 conversas validou esse procedimento (Seção 4.7).

4.6 Critérios de validação e correção

A comparação entre a documentação gerada pelos modelos e o gabarito ocorreu por correspondência semântica entre as regras. O processo não utilizou comparação textual literal, para que reformulações válidas da mesma lógica não fossem penalizadas. Antes da classificação, cada resposta foi mantida exatamente como salva, e seus itens foram numerados na ordem em que apareceram. No *prompt* C, apenas a lista final de regras passou por avaliação, sem a avaliação da seção de análise. O código-fonte constituiu a referência final do experimento: quando o comentário e o código divergiam, prevaleceu o comportamento do código.

Cada item passou por três perguntas avaliativas, aplicadas sempre nesta ordem, e recebeu uma única categoria. A primeira pergunta verificou se o item consiste em uma regra de negócio, com base nos mesmos critérios do gabarito. A segunda aferiu se o item está correto em relação ao código-fonte. A terceira checou se o item corresponde a uma regra do gabarito, com a mesma condição e a mesma consequência. As respostas a essas perguntas definiram as quatro categorias apresentadas a seguir:

- Fora do escopo: o item não se qualifica como regra de negócio, a exemplo da descrição de uma falha técnica do banco de dados. Essa categoria não entra no cálculo de nenhuma métrica.
- Alucinação: o item aborda uma regra de negócio, mas afirma um comportamento que o código não executa.

- ❏ Reformulação: o item está correto e corresponde a uma regra do gabarito, com a mesma condição e a mesma consequência, porém expressa em outras palavras.
- ❏ Enriquecimento: o item está correto, mas a regra reportada não consta do gabarito. Um exemplo é a detecção de lógicas em trechos desprovidos de comentário próprio.

A categoria fora do escopo foi acrescentada devido ao comportamento dos modelos nos testes dos *prompts*, nos quais foram listados tratamentos de erro mesmo sob instrução contrária. Alguns casos receberam tratamento específico. Um item gerado sem condição ou sem consequência foi classificado como fora do escopo. Uma regra repetida dentro da mesma resposta contou apenas uma vez. Um item que agrupasse duas regras distintas do gabarito foi desdobrado em duas entradas individuais. Quando o modelo gerou dois itens para a mesma regra do gabarito, a regra contou uma única vez na revocação, embora cada item correto tenha contado de forma independente na precisão. Nos casos em que o modelo estivesse correto e o gabarito equivocado, prevaleceria o código, e o gabarito seria corrigido em uma nova versão. Um item que corresponda a uma regra retirada do gabarito por incerteza sobre o comportamento do código (Seção 4.3) foi classificado como fora do escopo, sem efeito nas métricas.

A partir dessas categorias, calcularam-se a precisão, a revocação (*recall*), a métrica *F1 score* e a taxa de alucinação, definidas nas Equações 1 a 4. Nas fórmulas, R, E e A representam o número de reformulações, enriquecimentos e alucinações, respectivamente. A revocação corresponde à fração das regras do gabarito efetivamente encontradas pelo modelo. A taxa de alucinação, por sua vez, representa o complemento da precisão.

$$\text{Precisão} = \frac{R + E}{R + E + A} \tag{1}$$

$$\text{Revocação} = \frac{\text{regras do gabarito encontradas}}{\text{total de regras do gabarito}} \tag{2}$$

$$F_1 = \frac{2 \times \text{Precisão} \times \text{Revocação}}{\text{Precisão} + \text{Revocação}} \tag{3}$$

$$\text{Taxa de alucinação} = \frac{A}{R + E + A} \tag{4}$$

As métricas foram calculadas para cada combinação de modelo, *prompt* e rodada, com a soma dos itens gerados nos quatro programas. O resultado final de cada combinação é a média aritmética das três rodadas, acompanhada dos respectivos valores mínimo e máximo. Uma diferença de desempenho entre os modelos só foi considerada relevante quando superou essa margem de variação.

Para investigar a relação entre a ocorrência de alucinações e a complexidade estrutural, cada uma das 37 regras atômicas recebeu um de dois níveis antes do teste piloto. No nível linear, a regra depende de um único teste (um *IF* ou um *WHEN*), sem *GO TO*

no caminho até o teste. No nível não linear, a regra depende de dois ou mais testes combinados, como um *IF* dentro de outro ou condições agregadas com *AND* e *OR*, ou o seu caminho de execução passa por um *GO TO* antes do teste. Um *GO TO* que apenas encerra a seção após a consequência, comum nos programas de tela, não altera o nível, e as três regras procedurais do gabarito (Seção 4.3), alcançadas somente quando testes anteriores são satisfeitos, foram classificadas como não lineares. A reformulação herdou o nível da regra correspondente no gabarito, enquanto a alucinação e o enriquecimento receberam o nível do trecho de código a que se referem. A taxa de alucinação e a revocação foram calculadas por nível. Devido ao tamanho da amostra (37 regras), essa análise tem caráter exploratório.

As regras de contagem foram fixadas em 31/10/2026, antes da primeira resposta, e atualizadas em 09/11/2026, também antes do piloto, com o critério de complexidade e o tratamento dos itens que correspondem a regras retiradas por dúvida, com as datas registradas no repositório da pesquisa. Apenas essas regras poderiam ser ajustadas após o piloto, com registro de data e motivo, e os *prompts* não foram alterados. A classificação foi conduzida exclusivamente pelo discente.

Os resultados obtidos foram comparados de forma direta aos de Rajbhoj et al. (2025), que reportaram precisão média de 91%. O desempenho também foi contrastado com o da ferramenta COBRAIN (B S; CHIMALAKONDA, 2025), que atingiu precisão de 1,0 e revocação de 0,746 tendo como referência a saída da ferramenta baseada em regras *COBREX*, e F1 de 0,73 na avaliação com um gabarito construído manualmente. A comparação com o trabalho de Kalohia (2026) restringe-se à esfera qualitativa, pois esse estudo reporta a taxa de concordância com especialistas (93%), e não a precisão das regras extraídas.

4.7 Passos do desenvolvimento

A pesquisa seguiu as etapas listadas a seguir, detalhadas nas seções anteriores:

1. *Seleção de dados*: escolha de quatro programas da *CBSA*. Esses programas implementam as operações de débito, crédito e transferência nas camadas de tela e de processamento (Seção 4.2).

2. *Construção do gabarito*: extração automática dos pares de comentário e código e aplicação de filtro sintático. A etapa abrangeu a reavaliação manual dos itens descartados e a classificação com base nos critérios definidos com o orientador, o que resultou em 37 regras atômicas (Seção 4.3).

3. *Construção dos prompts*: especificação pelo discente, redação do texto final por um modelo não avaliado e testes em programas externos à amostra, até o congelamento da versão final (Seção 4.4).

4. *Preparação das entradas*: remoção dos comentários e das linhas de depuração. A verificação do código limpo ocorreu em duas camadas, acompanhada do registro do resumo criptográfico *SHA*-256 de cada entrada (Seção 4.5).

5. *Teste piloto*: execução única dos três *prompts* nos dois modelos. O piloto utilizou os programas *DBCRFUN* e *BNK1CRA*, que somam 17 das 37 regras, em um total de 12 conversas. O teste validou o procedimento de execução, as regras de contagem e o tempo de classificação. Suas respostas não compuseram os resultados finais nem alteraram os *prompts*.

6. *Execução*: realização de 72 conversas oficiais com os modelos *Claude Sonnet* 5.5 e *GPT-6* Sol, com três repetições por combinação (Seção 4.5).

7. *Avaliação semântica*: classificação individual de cada item das respostas pelo discente, com base nas três perguntas da Seção 4.6.

8. *Extração de métricas*: cálculo da precisão, da revocação, da métrica *F1 score* e da taxa de alucinação. A apuração ocorreu de forma global e segmentada por nível de complexidade, com o registro das médias e dos valores mínimo e máximo das rodadas.

9. *Análise comparativa*: confronto dos resultados obtidos entre os modelos, entre os *prompts* e em relação à literatura correlata.

O teste piloto empregou os programas *DBCRFUN* e *BNK1CRA* por dois motivos. Primeiro, são os menores programas da amostra. Segundo, ambos implementam a mesma operação de crédito e débito nas duas camadas, o que garantiu a passagem do teste pelos dois tipos de regra presentes no gabarito. Esses dois programas também integraram as execuções oficiais. Como cada submissão ocorreu em uma conversa nova e isolada, e as respostas do piloto foram descartadas, não houve transferência de contexto entre o teste e os resultados definitivos.

4.8 Uso de ferramentas de Inteligência Artificial

Ferramentas de Inteligência Artificial generativa foram empregadas como apoio na preparação da pesquisa e na redação do texto. O modelo *Claude Opus* 5.5, da *Anthropic*, redigiu uma versão preliminar das regras e das categorias dos pares candidatos do gabarito. Esse modelo também auxiliou na organização das planilhas, na especificação dos *prompts*, no rascunho das respostas esperadas dos exemplos do *prompt* B, nas regras de contagem e nos *scripts* de preparação. O modelo *Gemini 3.1 Pro*, do *Google*, usado por meio do *Gemini Notebook*, redigiu o texto final dos três *prompts* a partir dessa especificação, o que garantiu que nenhum dos modelos avaliados participasse da montagem do próprio

prompt. Na elaboração desta monografia, o *Claude Opus* 5.5 produziu rascunhos de seções a partir dos registros do trabalho, escreveu o código das figuras e apoiou a conferência das referências bibliográficas. O *Gemini Notebook* também foi usado na revisão de estilo do texto. Todo o material produzido com esse suporte passou por conferência do discente, com a validação de cada regra do gabarito no próprio código-fonte.

Por pertencer à mesma família de um dos modelos avaliados, o uso do *Claude Opus* 5.5 constitui uma ameaça à validade da pesquisa. Sua participação na especificação dos *prompts* recebeu a aprovação do orientador em 02/11/2026. A referida ameaça foi atenuada por três fatores: a adoção de um *prompt* único para ambos os modelos; a redação final das instruções por um modelo não avaliado; e a classificação das respostas de modo exclusivo pelo discente, por correspondência semântica, que independe da redação literal das regras. Além disso, a mesma conta do Claude foi usada no apoio à pesquisa e nas execuções do *Claude Sonnet* 5.5, que ocorreram em *chat* anônimo, fora de projetos e sem memória (Seção 4.5).

Capítulo 5

Conclusão

Até esta etapa, o trabalho definiu o procedimento de avaliação da documentação de regras de negócio gerada por *LLMs* a partir de código *COBOL* e preparou todos os seus elementos. Foram construídos o gabarito, com 37 regras atômicas extraídas de quatro programas da *CBSA*, os três *prompts*, congelados após testes em programas fora da amostra, e as regras de contagem usadas na comparação, fixadas antes da primeira resposta dos modelos.

Para a continuidade da pesquisa, pretende-se concluir as execuções oficiais com os modelos *Claude Sonnet* 5.5 e *GPT-6* Sol, classificar as respostas segundo as regras de contagem descritas no Capítulo 4 e calcular as métricas por modelo, por *prompt* e por nível de complexidade. Com esses dados, serão elaborados os capítulos de Resultados e Discussão e a Conclusão final.

A conclusão final retomará as perguntas de pesquisa e discutirá as limitações do trabalho, como o uso da interface de conversação sem controle de temperatura e a classificação por um único avaliador. Também apontará trabalhos futuros, como a aplicação do procedimento aos demais programas da *CBSA*.

Referências

ANTHROPIC. **Introducing Claude Sonnet 5.5**. 2026. Disponível em: <https://www.anthropic.com/claude-sonnet-5-5>. Acesso em: 9 nov. 2026. Citado na página 27.

B S, C.; CHIMALAKONDA, S. LLM vs rule-based - the COBRAIN tool and an empirical study on extracting business rules from COBOL. In: **Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering**. New York, NY, USA: Association for Computing Machinery, 2025. (EASE '25), p. 294–304. ISBN 9798400713859. Disponível em: <https://doi.org/10.1145/3756681.3756982>. Acesso em: 9 nov. 2026. Citado 7 vezes nas páginas 18, 23, 24, 25, 26, 32 e 37.

BANDARUPALLI, G. Code reborn AI-driven legacy systems modernization from COBOL to Java. **arXiv preprint arXiv:2504.11335**, 2025. Citado na página 11.

BENNETT, K. Legacy systems: coping with success. **IEEE Software**, v. 12, n. 1, p. 19–23, 1995. Citado na página 17.

BROWN, T. et al. Language models are few-shot learners. **Advances in neural information processing systems**, v. 33, p. 1877–1901, 2020. Citado 3 vezes nas páginas 19, 20 e 32.

CHEN, M. et al. Evaluating large language models trained on code. **arXiv preprint arXiv:2107.03374**, 2021. Citado na página 21.

CHIKOFSKY, E. J.; CROSS, J. H. Reverse engineering and design recovery: A taxonomy. **IEEE Software**, IEEE, v. 7, n. 1, p. 13–17, 1990. Citado na página 17.

CNBC. **New Jersey seeks COBOL programmers to fix unemployment system**. 2020. Disponível em: <https://www.cnbc.com/2020/04/06/new-jersey-seeks-cobol-programmers-to-fix-unemployment-system.html>. Acesso em: 29 jul. 2026. Citado na página 19.

DEVLIN, J. et al. **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**. 2019. Disponível em: <https://arxiv.org/abs/1810.04805>. Acesso em: 9 nov. 2026. Citado na página 19.

DIGGS, C. et al. Leveraging LLMs for legacy code modernization: Evaluation of LLM-generated documentation. In: **2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)**. [S.l.: s.n.], 2025. p. 177–184. Citado 4 vezes nas páginas 12, 18, 23 e 25.

GRAHL, J. K. **Data-driven decision making in software modernization**. Dissertação (Mestrado) — RWTH Aachen University, Aachen, 2026. Disponível em: <https://doi.org/10.18154/RWTH-2026-04376>. Acesso em: 9 nov. 2026. Citado na página 28.

IBM. **Generating information about procedures**. [2016?]. Enterprise COBOL for z/OS 6.1: Programming Guide. Disponível em: <https://www.ibm.com/support/knowledgecenter/SS6SG3_6.1.0/pg/tasks/tpbug10.html>. Acesso em: 9 nov. 2026. Citado na página 34.

_____. **SQLCODE**. [2016?]. Db2 12 for z/OS: SQL Reference. Disponível em: <https://www.ibm.com/docs/SSEPEK_12.0.0/sqlref/src/tpc/db2z_sqlcode.html>. Acesso em: 10 nov. 2026. Citado na página 16.

_____. **CICS Db2 program preparation**. [2018?]. CICS Transaction Server for z/OS 5.5. Disponível em: <https://www.ibm.com/support/knowledgecenter/SSGMCP_5.5.0/applications/deploying/dfhtk20.html>. Acesso em: 9 nov. 2026. Citado 2 vezes nas páginas 16 e 29.

_____. **CICS Banking Sample Application (CBSA)**. 2023. Repositório GitHub ijmitch/cics-banking-sample-application-cbsa, commit e224756. Disponível em: <https://github.com/ijmitch/cics-banking-sample-application-cbsa>. Acesso em: 9 nov. 2026. Citado 4 vezes nas páginas 14, 16, 28 e 31.

_____. **Enterprise COBOL for z/OS 6.4: Language Reference**. [S.l.], 2024. SC27-8713-03. Disponível em: <https://publibfp.dhe.ibm.com/epubs/pdf/igy6lr40.pdf>. Acesso em: 10 nov. 2026. Citado 2 vezes nas páginas 15 e 16.

_____. **Relatório Anual 2025**. Armonk, NY, 2025. Disponível em: <https://www.ibm.com/investor/services/annual-report>. Acesso em: 21 jul. 2026. Citado na página 11.

KALOHIA, A. Leveraging generative AI for extracting business requirements from legacy COBOL and PL/I code. In: LI, Y.; REHM, G.; TU, M. (Ed.). **Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)**. San Diego, California, USA: Association for Computational Linguistics, 2026. p. 43–55. ISBN 979-8-89176-394-4. Disponível em: <https://aclanthology.org/2026.acl-industry.4/>. Acesso em: 9 nov. 2026. Citado 5 vezes nas páginas 12, 23, 25, 26 e 37.

KAPLAN, J. et al. **Scaling Laws for Neural Language Models**. 2020. Disponível em: <https://arxiv.org/abs/2001.08361>. Acesso em: 9 nov. 2026. Citado na página 19.

KHEMKA, S.; MAJUMDAR, A. Legacy modernization with AI–mainframe modernization. **arXiv preprint arXiv:2512.05375**, 2025. Citado na página 11.

LEI, F. et al. **Enhancing COBOL Code Explanations: A Multi-Agents Approach Using Large Language Models**. 2025. ArXiv preprint arXiv:2507.02182. Disponível em: <https://arxiv.org/abs/2507.02182>. Acesso em: 9 nov. 2026. Citado 2 vezes nas páginas 29 e 32.

LI, R. et al. StarCoder: may the source be with you! **arXiv preprint arXiv:2305.06161**, 2023. Citado na página 21.

MINAEE, S. et al. Large language models: A survey. **arXiv preprint arXiv:2402.06196**, 2024. Citado na página 20.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **FIPS PUB 180-4: Secure Hash Standard (SHS)**. Gaithersburg, MD, 2015. Citado na página 34.

NIJKAMP, E. et al. CodeGen: An open large language model for code with multi-turn program synthesis. **arXiv preprint arXiv:2203.13474**, 2022. Citado na página 21.

OPENAI. **ChatGPT: Optimizing Language Models for Dialogue**. 2022. Disponível em: <https://openai.com/blog/chatgpt/>. Acesso em: 30 jul. 2026. Citado na página 20.

_____. GPT-4 technical report. **arXiv preprint arXiv:2303.08774**, 2023. Citado na página 20.

_____. **GPT-6 and other models in ChatGPT**. 2026. OpenAI Help Center. Disponível em: <https://help.openai.com/en/articles/11909943>. Acesso em: 17 out. 2026. Citado na página 28.

_____. **Introducing GPT-6 Sol and Luna**. 2026. Disponível em: <https://openai.com/index/introducing-gpt-6-sol-and-luna/>. Acesso em: 9 nov. 2026. Citado na página 27.

OUYANG, L. et al. Training language models to follow instructions with human feedback. **arXiv preprint arXiv:2203.02155**, 2022. Citado na página 20.

RADFORD, A. et al. Improving language understanding by generative pre-training. San Francisco, CA, USA, 2018. Citado na página 20.

RAJBHOJ, A. et al. Leveraging LLM for software modernization: COBOL functionality extraction case study. In: IEEE. **2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)**. [S.l.], 2025. p. 14–21. Citado 8 vezes nas páginas 11, 12, 18, 22, 24, 25, 26 e 37.

RAYNS, C. et al. **CICS Transaction Server from Start to Finish**. IBM Redbooks, 2011. SG24-7952-00. Disponível em: <https://www.redbooks.ibm.com/abstracts/sg247952.html>. Acesso em: 10 nov. 2026. Citado na página 16.

REUTERS. **Banks scramble to fix old systems as IT 'cowboys' ride into sunset**. 2017. Disponível em: <https://www.reuters.com/article/us-usa-banks-cobol-idUSKBN17C0D8/>. Acesso em: 29 jul. 2026. Citado na página 18.

ROY, D.; FAKHOURY, S.; ARNAOUDOVA, V. Reassessing automatic evaluation metrics for code summarization tasks. In: **Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '21)**. New York, NY, USA: ACM, 2021. p. 1105–1116. Citado 2 vezes nas páginas 12 e 26.

SABETTO, R. et al. **Impact of Comments on LLM Comprehension of Legacy Code**. 2025. Disponível em: <https://arxiv.org/abs/2506.11007>. Acesso em: 9 nov. 2026. Citado 2 vezes nas páginas 24 e 25.

SAMMET, J. E. The early history of COBOL. In: WEXELBLAT, R. L. (Ed.). **History of Programming Languages**. New York, NY, USA: Association for Computing Machinery, 1978. p. 199–243. ISBN 0127450408. Disponível em: <https://doi.org/10.1145/800025.1198367>. Acesso em: 9 nov. 2026. Citado na página 15.

SEBESTA, R. W. **Conceitos de Linguagens de Programação**. 11. ed. Porto Alegre: Bookman, 2018. Citado na página 17.

SOCIEDADE BRASILEIRA DE COMPUTAÇÃO. **Código de Conduta para Autores em Publicações da SBC**. 2024. Aprovado pela Resolução n. 01, de 20 de março de 2024. Disponível em: <https://www.sbc.org.br/wp-content/uploads/2024/07/Revis-o-e-atualiza-o-do-C-digo-de-Conduta-para-Autores_Resolucao_2024-001-1.pdf>. Acesso em: 10 nov. 2026. Citado na página 4.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Addison Wesley, 2011. Citado 3 vezes nas páginas 15, 17 e 18.

TOUVRON, H. et al. LLaMA: Open and efficient foundation language models. **arXiv preprint arXiv:2302.13971**, 2023. Citado na página 20.

UNITED STATES. Government Accountability Office. **Information Technology: Agencies Need to Develop and Implement Modernization Plans for Critical Legacy Systems**. Washington, D.C., 2019. Relatório Governamental. Citado na página 18.

WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. In: **Advances in Neural Information Processing Systems**. Red Hook, NY, USA: Curran Associates, Inc., 2022. v. 35, p. 24824–24837. Disponível em: <https://proceedings.neurips.cc/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html>. Acesso em: 9 nov. 2026. Citado 3 vezes nas páginas 19, 32 e 33.

Apêndices

APÊNDICE A

Texto dos *prompts*

Os três *prompts*, na versão congelada em 24/10/2026 (Seção 4.4), compartilham a mesma base (Seção A.1) e o mesmo fechamento (Seção A.2). O *prompt* A é formado pela base seguida do fechamento. Nos *prompts* B e C, o bloco da estratégia (Seções A.3 e A.4) é inserido entre a base e o fechamento.

No fechamento, o marcador `{{CODIGO_COBOL}}` é substituído pelo código do programa sem comentários e sem linhas de depuração (Seção 4.5). O texto é reproduzido sem alterações, inclusive os trechos em inglês do código e das mensagens dos exemplos.

A.1 Base comum aos três *prompts*

```
Papel: Analista de sistemas legados que documenta as regras de negócio de programas
    COBOL para a área de negócio de um banco.
Tarefa: Ler o programa COBOL fornecido e listar as regras de negócio que ele
    implementa.
Definição de regra de negócio: Tudo o que a área de negócio (gerente ou cliente do
    banco) entende. Detalhes de código que o pessoal de negócio não entenderia ficam
    de fora.

O que não listar:
- Tratamento de erro padrão e falhas técnicas de banco de dados ou de sistema
    (códigos de retorno, ABEND, rollback);
- Gravação de registros técnicos, de log ou de auditoria;
- Controle de concorrência e ordem técnica de atualização;
- Navegação de tela e teclas de função;
- Validação de formato do campo digitado: se o conteúdo é numérico, se há espaços no
    meio, se há mais de um ponto decimal, se o caractere de sinal é válido.

O que listar, mesmo sendo checagem de campo:
- A obrigatoriedade de um dado (o campo não pode ficar vazio);
- Restrições sobre o valor em si (por exemplo: limites mínimos ou máximos, ou a
    lista de valores aceitos para um campo).
```

```
Quando um mesmo teste cobre ausência e formato, a classificação segue o que o código
    testa: teste de tamanho zero ou de campo só com espaços é obrigatoriedade
    (listar); teste de valor numérico é formato (não listar), mesmo que a mensagem
    exibida peça para preencher o campo.

Escopo: liste apenas regras que dependem de uma decisão no código (IF, EVALUATE ou
    teste de um código de retorno). Cada regra corresponde a um caminho dessa
    decisão. Não liste atribuições de valores, preparação de dados para gravação ou
    envio, nem exibição de dados, quando não houver uma decisão associada. Dentro de
    um mesmo caminho de decisão, não crie uma regra para cada campo atribuído ou
    exibido: descreva a consequência principal.

Agrupamento: valores alternativos testados numa mesma decisão que levam à mesma
    consequência formam uma única regra. Em decisões aninhadas, cada regra descreve
    o caminho completo até a consequência final; não liste como regra um teste
    intermediário que apenas leva a outro teste.

Atomicidade: Uma regra por item, com uma condição e uma consequência. Uma regra
    composta deve ser separada em itens.
Ocorrências repetidas: Se a mesma regra aparece em mais de um ponto do código, cada
    ocorrência é listada separadamente, com o seu parágrafo.
Fidelidade: Listar só o que o código de fato implementa, e não o que seria esperado
    de um sistema bancário. Se o comportamento for ambíguo, dizer isso no item.
Linguagem: Cada regra é uma frase em linguagem de negócio, em português. A frase
    pode citar entre parênteses o campo, a condição ou o código do programa que
    sustenta a regra (por exemplo, o nome do campo testado ou o valor de um código
    de retorno).

Formato da resposta, sem nenhum texto antes ou depois da lista:
1. Regra: <frase em linguagem de negócio>
   Condição: <quando se aplica>
   Consequência: <o que acontece>
   Parágrafo: <nome do parágrafo ou seção COBOL>
```

A.2 Fechamento comum aos três *prompts*

```
<codigo>
{{CODIGO_COBOL}}
</codigo>

Lembrete: a resposta deve seguir estritamente o formato "1. Regra: ... Condição: ...
    Consequência: ... Parágrafo: ...", sem nenhum texto antes ou depois da lista
    final.
```

A.3 Bloco do *prompt* B (*few-shot*)

```
Para referência, veja abaixo dois exemplos de como executar a tarefa.

Exemplo 1:

Código:
       PROCEDURE DIVISION.
       PREMIERE SECTION.
       A010.

           MOVE SORTCODE TO COMM-SCODE
                            DESIRED-SORT-CODE.

           MOVE SPACES TO WS-UNSTR-TITLE.
           UNSTRING COMM-NAME DELIMITED BY SPACE
              INTO WS-UNSTR-TITLE.

           MOVE ' ' TO WS-TITLE-VALID.

           EVALUATE WS-UNSTR-TITLE
              WHEN 'Professor'
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Mr       '
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Mrs      '
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Miss     '
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Ms       '
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Dr       '
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Drs      '
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Lord     '
                 MOVE 'Y' TO WS-TITLE-VALID

              WHEN 'Sir      '
                 MOVE 'Y' TO WS-TITLE-VALID
```

```
           WHEN 'Lady     '
              MOVE 'Y' TO WS-TITLE-VALID

           WHEN '         '
              MOVE 'Y' TO WS-TITLE-VALID

           WHEN OTHER
              MOVE 'N' TO WS-TITLE-VALID
        END-EVALUATE.

        IF WS-TITLE-VALID = 'N'
          MOVE 'N' TO COMM-UPD-SUCCESS
          MOVE 'T' TO COMM-UPD-FAIL-CD
          GOBACK
        END-IF

        PERFORM UPDATE-CUSTOMER-VSAM
        PERFORM GET-ME-OUT-OF-HERE.

    A999.
        EXIT.


    UPDATE-CUSTOMER-VSAM SECTION.
    UCV010.

        MOVE COMM-CUSTNO TO DESIRED-CUSTNO.

        EXEC CICS READ FILE('CUSTOMER')
             RIDFLD(DESIRED-CUST-KEY)
             INTO(WS-CUST-DATA)
             UPDATE
             RESP(WS-CICS-RESP)
             RESP2(WS-CICS-RESP2)
        END-EXEC.

        IF WS-CICS-RESP NOT = DFHRESP(NORMAL)

           MOVE 'N' TO COMM-UPD-SUCCESS

           IF WS-CICS-RESP = DFHRESP(NOTFND)
              MOVE '1' TO COMM-UPD-FAIL-CD
           ELSE
              MOVE '2' TO COMM-UPD-FAIL-CD
           END-IF
```

```
              GO TO UCV999

           END-IF.

Resposta esperada:
1. Regra: Ao atualizar os dados do cliente, o título (primeira palavra do nome) deve
    ser um dos aceitos: Professor, Mr, Mrs, Miss, Ms, Dr, Drs, Lord, Sir ou Lady, ou
    pode ficar em branco. Com outro título, a atualização não é feita e é sinalizada
    como malsucedida com o código de falha 'T'.
   Condição: a primeira palavra do nome informado não é um dos títulos aceitos nem
    está em branco
   Consequência: a atualização não é feita; indicador de sucesso 'N' e código de
    falha 'T'
   Parágrafo: A010 (PREMIERE SECTION)

2. Regra: Se o cliente a ser atualizado não for encontrado (resposta NOTFND da
    leitura), a atualização é sinalizada como malsucedida com o código de falha '1'.
   Condição: a leitura do registro do cliente retorna NOTFND
   Consequência: indicador de sucesso 'N' e código de falha '1'
   Parágrafo: UCV010 (UPDATE-CUSTOMER-VSAM)

Exemplo 2:

Código:
       EDIT-DATA SECTION.
       ED010.
           IF ACCNOI NOT NUMERIC
              MOVE 'Please enter an account number.   ' TO
                 MESSAGEO
              MOVE 'N' TO VALID-DATA-SW
           END-IF.

       ED999.
           EXIT.


       VALIDATE-DATA SECTION.
       VD010.
           IF ACTYPEI NOT = 'CURRENT ' AND
           ACTYPEI NOT = 'SAVING  ' AND
           ACTYPEI NOT = 'LOAN    ' AND
           ACTYPEI NOT = 'MORTGAGE' AND
           ACTYPEI NOT = 'ISA     '

              MOVE SPACES TO MESSAGEO
              STRING 'Account Type must be CURRENT, SAVING, LOAN, '
                 'MORTGAGE or ISA' DELIMITED BY SIZE,
```

```
                ', Then press PF5.' DELIMITED BY SIZE
             INTO MESSAGEO
             MOVE -1 to ACTYPEL
             MOVE 'N' TO VALID-DATA-SW
          END-IF.

          IF INTRTL = ZERO
             MOVE SPACES TO MESSAGEO
             STRING 'Please supply a numeric interest rate '
                'then press PF5.' DELIMITED BY SIZE
                INTO MESSAGEO
             MOVE 'N' TO VALID-DATA-SW
             MOVE -1 TO INTRTL
             GO TO VD999
          END-IF.

Resposta esperada:
1. Regra: Se o tipo de conta informado não for CURRENT, SAVING, LOAN, MORTGAGE ou
    ISA, a entrada é considerada inválida e é exibida a mensagem 'Account Type must
    be CURRENT, SAVING, LOAN, MORTGAGE or ISA. Then press PF5.'
   Condição: tipo de conta diferente de CURRENT, SAVING, LOAN, MORTGAGE e ISA
   Consequência: entrada inválida, com a mensagem indicada
   Parágrafo: VD010 (VALIDATE-DATA)

2. Regra: Se a taxa de juros não for informada (campo vazio), a entrada é
    considerada inválida e é exibida a mensagem 'Please supply a numeric interest
    rate then press PF5.'
   Condição: campo de taxa de juros sem conteúdo (INTRTL igual a zero)
   Consequência: entrada inválida, com a mensagem indicada; as demais validações não
    são feitas
   Parágrafo: VD010 (VALIDATE-DATA)

Abaixo encontra-se o código que você deve analisar agora.
```

A.4 Bloco do *prompt* C (*chain-of-thought*)

```
Instrução especial para esta execução:
Sua resposta deve ter duas seções, com os títulos exatos "Análise" e "Regras de
    negócio", nesta ordem.
A regra acima de não haver texto antes ou depois da lista aplica-se à seção "Regras
    de negócio", que vem por último. A seção "Análise" vem antes da lista.

A seção "Análise" deve conter:
1. os parágrafos e o fluxo principal do programa;
2. as estruturas de decisão (IF, EVALUATE, testes de código de retorno);
3. a classificação de cada estrutura como negócio, infraestrutura ou interface;
```

```
4. a condição e a consequência das estruturas de negócio.

A seção "Regras de negócio" contém só as regras de negócio, no formato definido
    acima.
```